TRABAJO FIN DE GRADO

Pablo Martin Tomasi
Segundo de DAM



1 DE JUNIO DE 2026

TUTORA:
Amelia González

ÍNDICE

Introducción 2
Justificación del proyecto 2
Objetivos 2
Desarrollo 2
Fundamentación teórica 3
Materiales y métodos 5
Diagrama de casos de uso debidamente documentado 5
Diseño lógico/conceptual de la base de datos 6
Prototipo inicial 6
Diagramas de flujo de los procesos más importantes 10
Descripción de los componentes principales del software 15
Resultados y análisis 27
Resultados 27
Análisis 45
Conclusiones 46
Líneas de investigación futuras 46
Bibliografía 48

Introducción

Descubre tu alrededor, es mi TFG, es una aplicación que he estado desarrollando durante el segundo año de grado. Y es una web que está orientada para acercar el senderismo a cualquier persona, sin importar la experiencia que tengan o la ubicación en la que se encuentren.

El objetivo principal es que la web es bastante intuitiva, ya que tiene integrado un mapa interactivo donde tiene todas las rutas disponibles, también ese mapa te posiciona siempre en tu ubicación, pues eso puede ser bastante practico para que la gente pueda ver de una mejor forma las rutas que tienen por su ubicación. Además de eso hay un catálogo con todas las rutas disponibles, y unos filtros para poder filtrar por dificultad, tipo de ruta y dirección.

Justificación del proyecto

La elección de hacer este proyecto es dada a una motivación personal por el senderismo, ya que es una actividad que intento practicar habitualmente. Y he llegado a considerar que merece poder hacer una plataforma moderna, accesible y social.

Desde la utilidad que puede recibirá Descubre tu alrededor, es orientarla a un amplio colectivo, para los senderistas habituales que desean crear sus propias rutas o enseñar a los usuarios de la web las rutas que ellos aman. Como también para personas que inician en el mundo del senderismo poder buscar rutas cerca de su ubicación. Además, la web va a resultar útil para los viajeros que desean explorar la naturaleza cuando se van de viaje, ya que gracias al mapa interactivo pueden descubrir rutas que hay alrededor de ellos.

Además de eso yo opino que mi web es superior a las demás, ya que en el mercado webs de senderismo como Wkiloc, AllTrails o Visorando, ofrecen catálogos extensos de rutas de senderismo, pero lo que me he llegado a percatar es que ninguna de esas webs no ofrece una opción para poder realizar quedadas para hacer las rutas con más gente. Por eso Descubre tu alrededor cubre esta necesidad con una funcionalidad específica de organización de salidas, fomentando así la vertiente social y comunitaria del senderismo.

Objetivos

- **Objetivos generales**
  - Desarrollar una plataforma web que permita a los usuarios crear rutas de senderismo, explorar rutas de senderismo cercanas mediante un mapa interactivo, y poder integrar una herramienta que permita tener una interacción social entre usuarios
- **Objetivos específicos**
  - Permitir a los usuarios crear, editar y compartir sus propias rutas
  - Implementar un sistema de comentarios vinculado al historial de rutas completadas
  - Desarrollar un sistema de quedadas para fomentar la interacción entre usuarios
  - Incorporar un sistema de filtros para facilitar la búsqueda de rutas
  - Implementar un panel de administración para la gestión de reportes

Desarrollo

Fundamentación teórica

La arquitectura de mi proyecto es la de MVC, ya que puedo llegar a tener un enfoque bastante amplio pudiendo utilizarlo en el desarrollo web ya que me permite separar la lógica del negocio, la presentación y el control del flujo de datos. Y NodeJS me puede llegar a permitir la facilitación de ese patrón.

Las tecnologías que he utilizado son las siguientes:

- **NodeJS**, es un entorno de ejecución de JS en el lado del servidor. Es muy ideal ya que puedo llegar a hacer fronted y backend sin tener que cambiar de lenguaje. Y con eso se puede llegar a agilizar el desarrollo de una mejor forma.



- **Express**, es un framwork minimalista para NodeJS que puede ayudar a facilitar la creación de los servidores web, y poder hacer la gestión de las rutas y de las peticiones



- **PostgreSQL**, es la base de datos que he estado utilizando, ya la he estado usando ya que es compatible con la biblioteca de PostGIS, que me ayuda para poder tener los datos geoespaciales, y con esa extensión he podido hacer de buena forma la gestión y almacenamiento de las coordenadas para las rutas de senderismo



- **Handlebars**, es la plantilla que he estado utilizando para el fronted para poder generar la vista html de buena forma



- **CSS,** es el lenguaje de estilo que he estado utilizando para poder hacer todo el diseño bonito del html. Y luego también he estado utilizando la biblioteca de boostrap para tener mejores diseños



- **Leaflet,** he estado usando este api, ya que el api de Google maps tiene un uso limitado. Y luego con este api me permite hacer lo mismo que el api de Google maps y gratis. La

he estado usando para poder hacer las rutas, donde el usuario usa el mapa interactivo para poder hacer las rutas



- **Nominatim,** este api la he estado utilizando para la dirección de las rutas, cuando el usuario hace clic en la zona de donde empieza el mapa, y luego en los detalles de las rutas se verá la dirección de la ruta

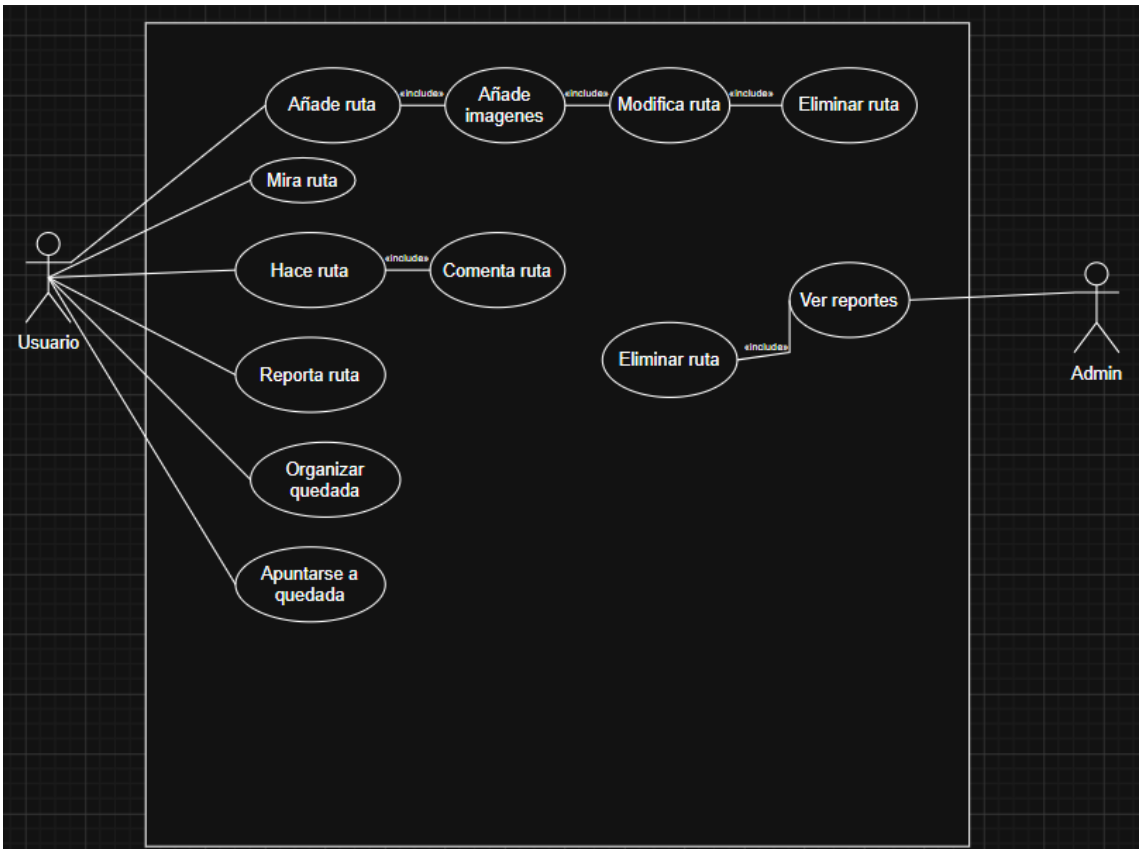


- **Visual studio code,** es el editor de código que he estado utilizando para poder hacer la web, lo he estado usando ya que tiene buen soporte para NodeJS



Materiales y métodos

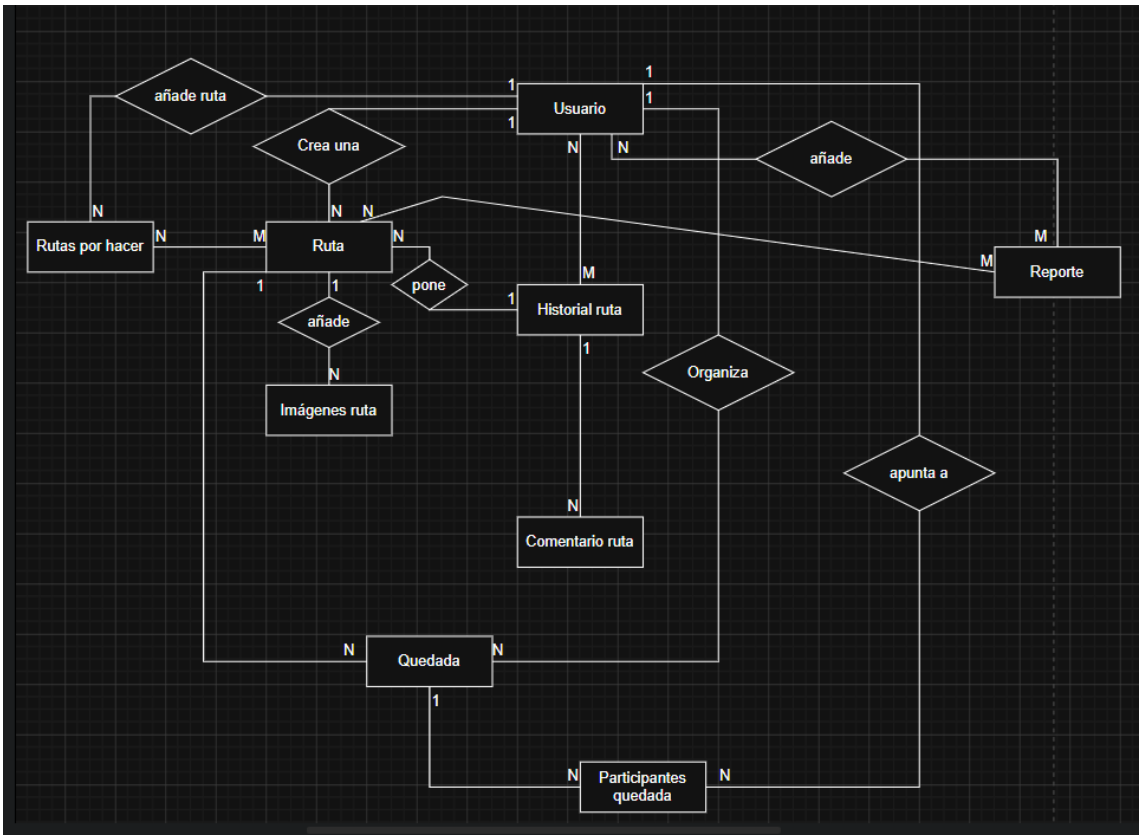
Diagrama de casos de uso debidamente documentado



En esta app, en el perfil hay 2 tipos.

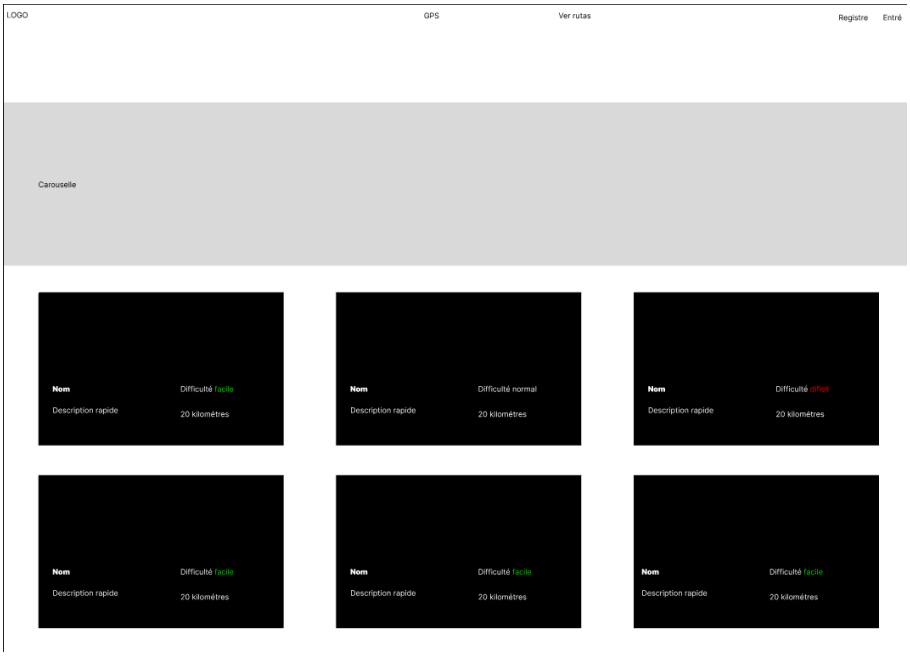
- Usuario
    - Este es el perfil general de la aplicación. Ya que el usuario puede añadir rutas, añadir imágenes en las rutas que han creado, y modificar y eliminar las rutas que han creado. Además, los usuarios pueden ver el listado de las rutas disponibles, marcar rutas como hechas y añadir un comentario en ella. Reportar rutas por si hay algún error, y poder organizar o apuntarse a una quedada de una ruta.
- Admin
    - En el perfil del admin, es la gestión de los reportes, donde puede ver que rutas han sido reportadas. Y además puede modificar los datos de las rutas por si hay varios errores

Diseño conceptual de la base de datos



Prototipo inicial

- Vista index



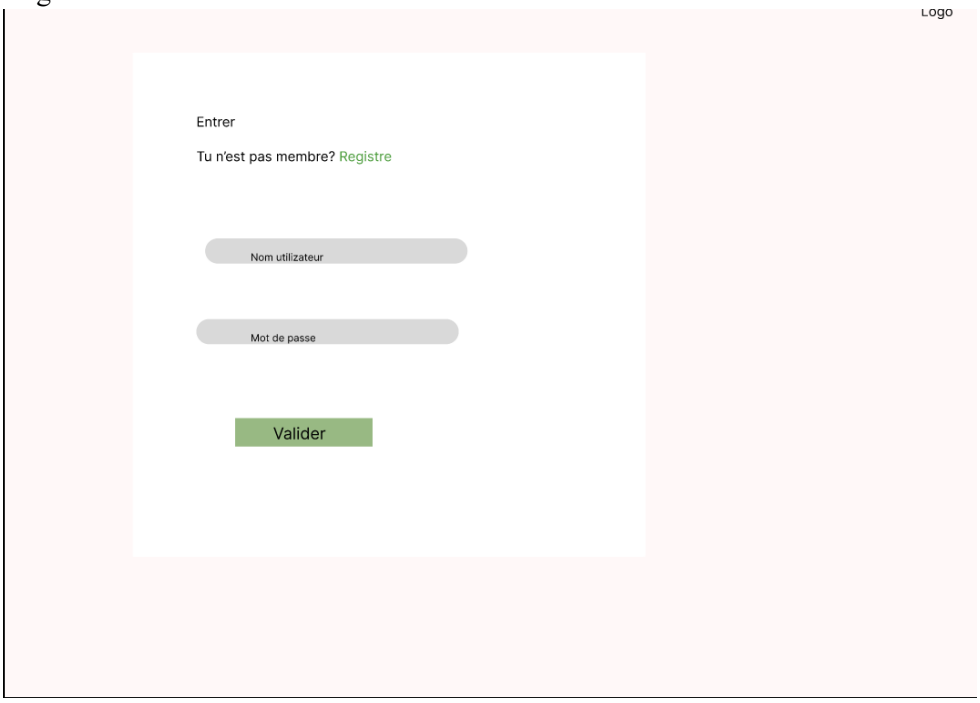
Esta sería la vista index, con un carousel, y unas rutas aleatorias debajo del carousel

- Detalle ruta



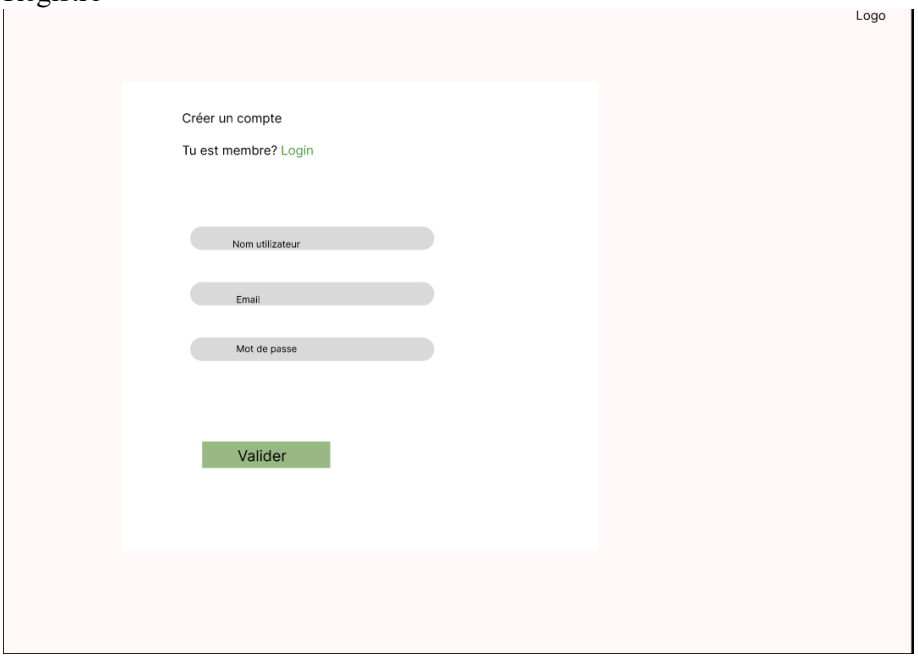
Detalle de la ruta, esta vista era la vista que tenía pensada en un principio

- Login



Esta era la vista que tenía prevista en un principio para el formulario del inicio de sesión

- Registro



Esta era la vista para el formulario de registro que tenia de idea en un principio

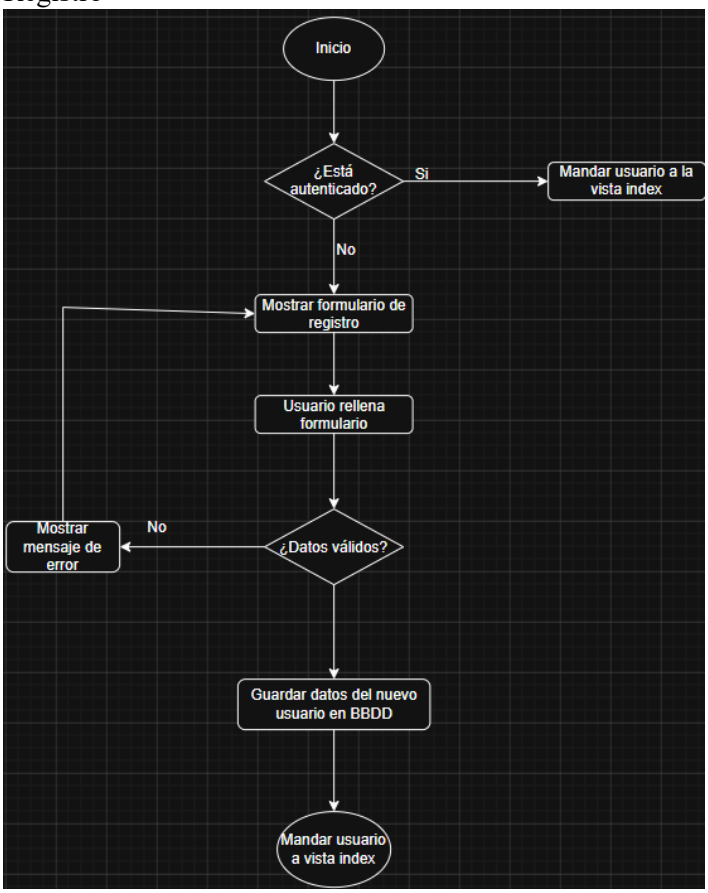
- Perfil usuario



Esta es la vista que tenía prevista en un principio con la vista del perfil

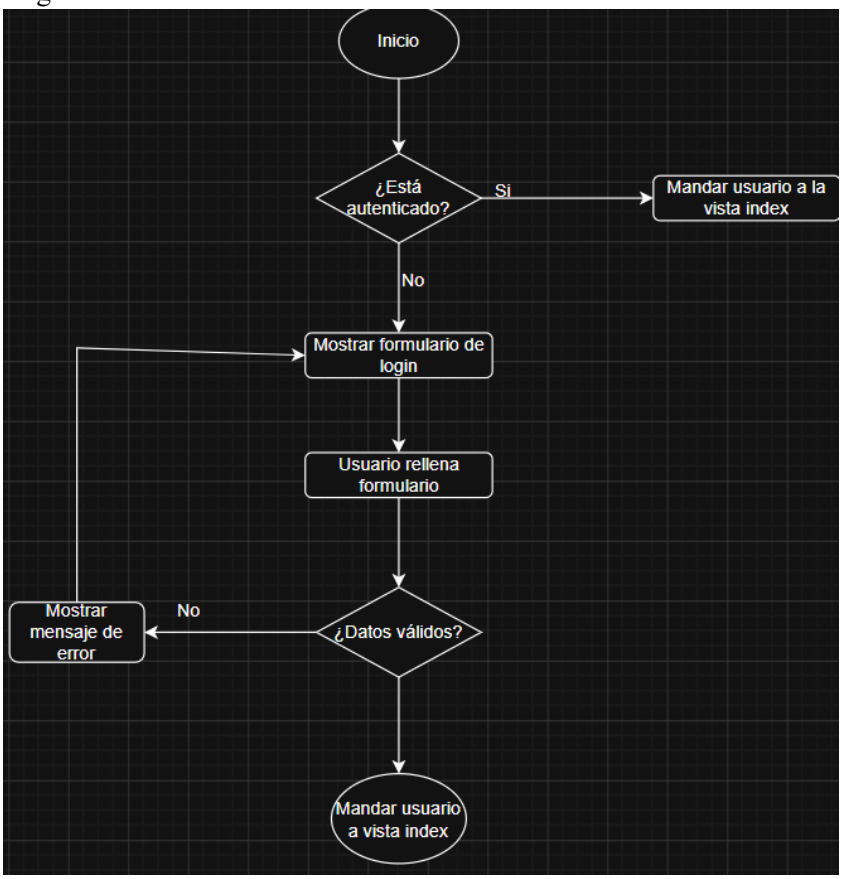
Diagramas de flujo de los procesos más importantes

- Registro



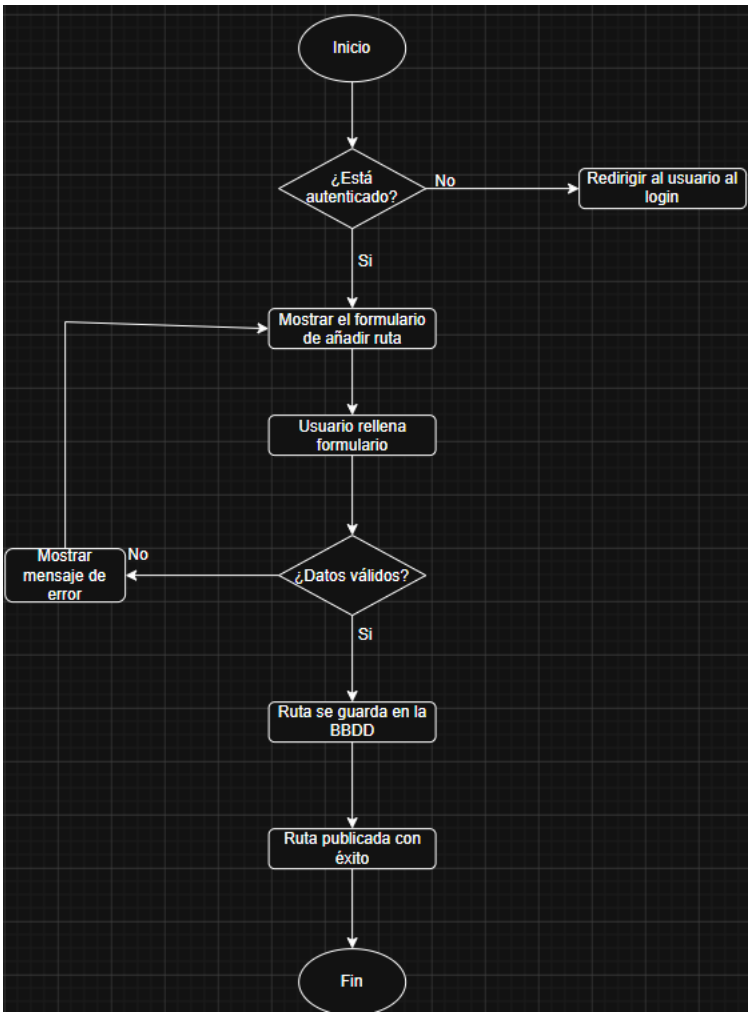
Cuando se ejecuta la web, y el usuario quiere registrarse primero se verifica si esta autenticado, si lo esta le manda a la vista index. Pero si no lo esta le muestra el formulario de registro. El usuario rellena el formulario, si hay un error le vuelve a mostrar el formulario. Pero si funciona, agrega al nuevo usuario a dentro de la base de datos, y luego le manda a la vista index.

- Login



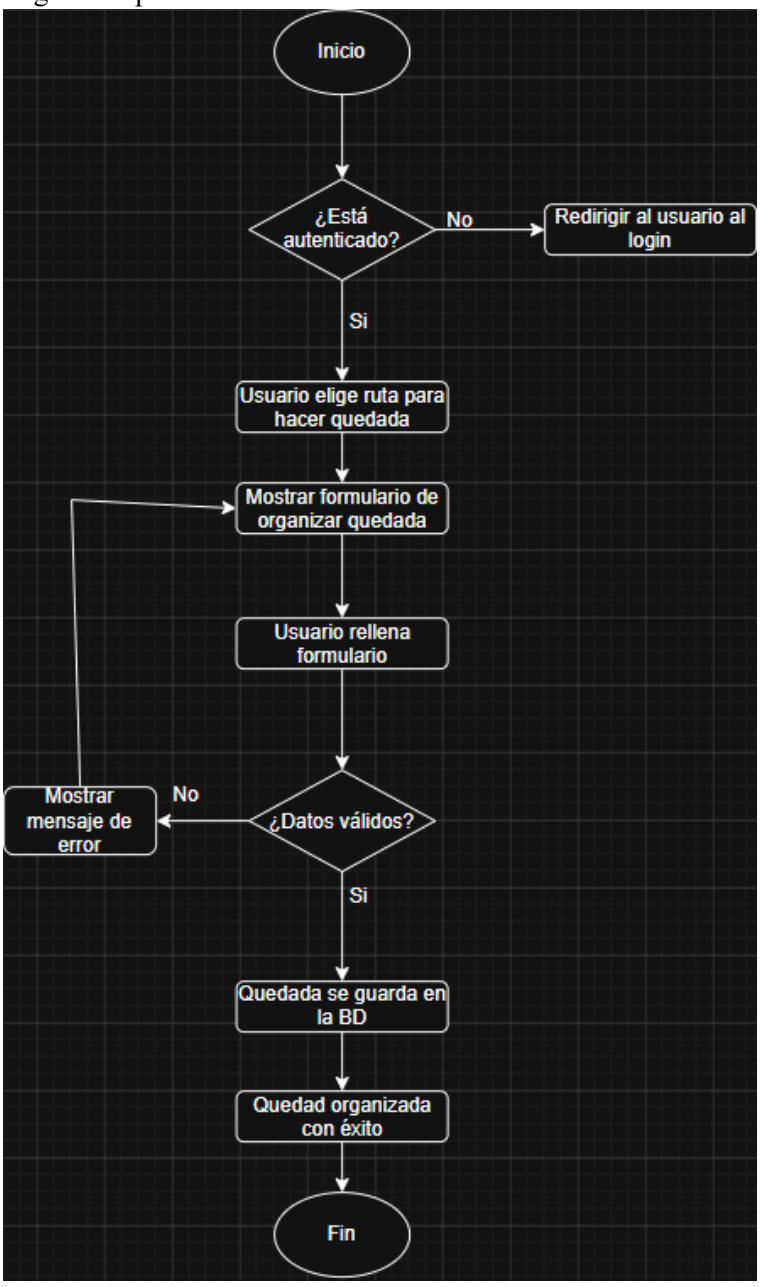
Si el usuario esta autenticado, le manda a la vista index. Pero si no los esta le muestra el formulario del login. Luego el usuario rellena el formulario, se verifica si los datos son válidos. Si no son validos le muestra el mensaje de error le enseña de nuevo el formulario. Si inicia sesión de manera correcta le manda al usuario a la vista index.

- Añadir ruta



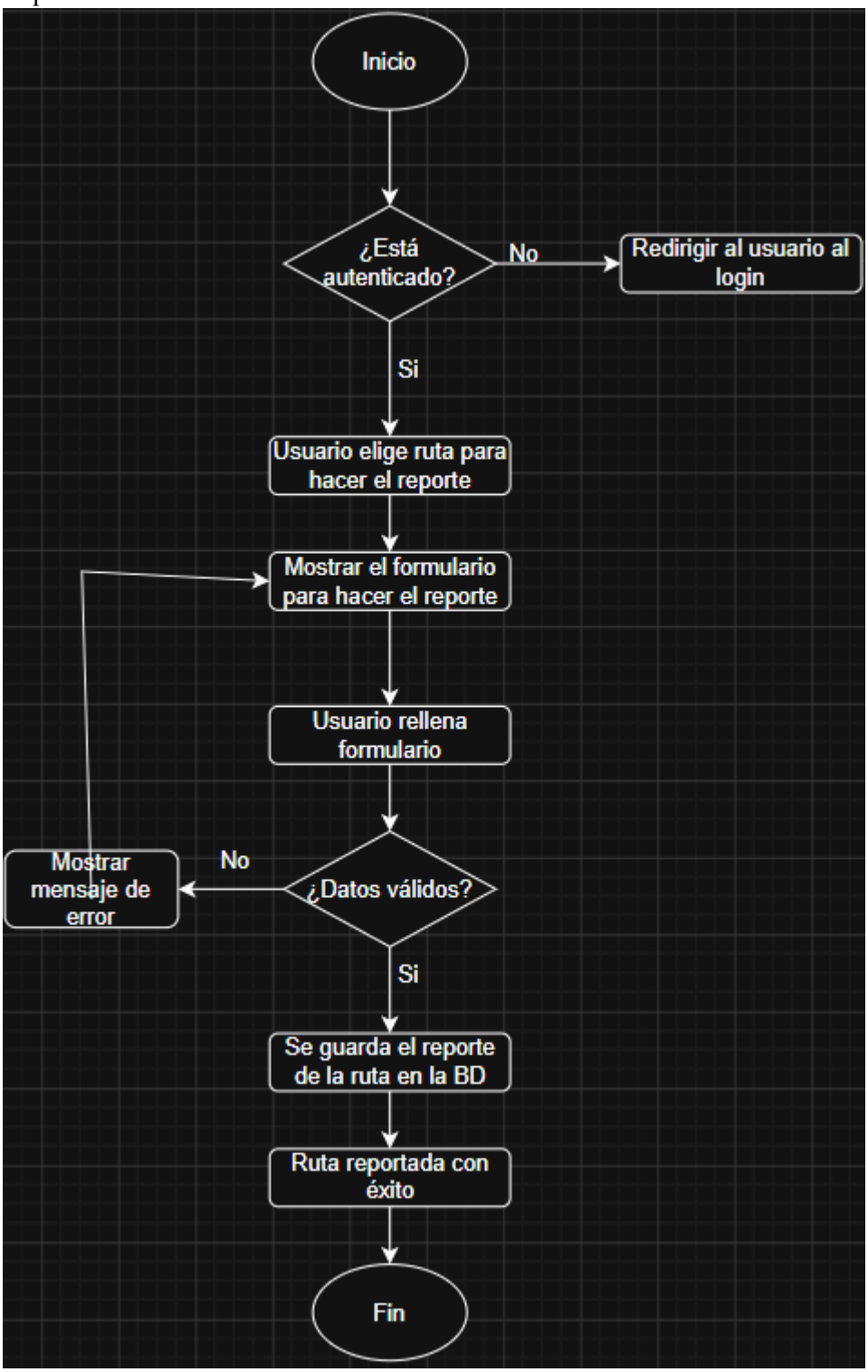
Si el usuario no esta autenticado le manda a la vista de login para que inicie sesión. Pero si el usuario esta autenticado, le muestra el formulario para añadir la nueva ruta. Luego el usuario rellena el formulario, se verifican si los datos son válidos. Si no son validos le vuelve a enseñar el formulario. Pero si son válidos, se guarda la ruta con éxito en la base de datos, y luego se vuelve a la vista de verRutas

- Organizar quedada



Primero se verifica si el usuario esta autenticado, si no lo esta le manda al formulario del login. Pero si lo está, el usuario elige una ruta. Al elegir la ruta se le muestra el formulario. Luego tiene que ingresar los datos, si hay algún dato que falla le vuelve a mostrar el formulario. Pero si están bien los datos, se añade la quedada a la base de datos, y se organiza la quedada con éxito.

- Reportar ruta



Se verifica que el usuario esta autenticado, si no lo esta le manda a la vista del login. Pero si lo está, se tiene que elegir la ruta que se desea reportar. Luego le muestra el formulario para hacer el reporte, si el usuario al mandarlo hay un error le vuelve a enseñar el formulario. Pero si no hay se guarda el reporte de en la base de datos,

Descripción de los componentes principales del software

Registro del usuario:

Cuando el usuario carga la vista de registro, se le enseña la vista del formulario del registro. Donde el usuario tendrá que ingresar su email, nombre de usuario y su contraseña. Al enviar el formulario, los datos se van a transmitir mediante una petición POST al controlador de LoginController.js.

Antes de añadir al nuevo usuario en la base de datos, el controlador hace una sentencia select, buscando a un usuario con el mismo email que ha puesto el usuario, normalmente no debe de haber ningún usuario con ese email. Pero si hay un usuario con ese email, se ve un mensaje de error diciendo que ya existe un usuario con ese email. Y el usuario tendrá que modificar su email si se ha equivocado.

Si los datos son válidos, luego la contraseña se encripta mediante la biblioteca de NodeJS de bcrypt, gracias a esto se puede garantizar de que la contraseña del usuario no se guarde en texto plano en la base de datos, manteniendo mayor seguridad para el usuario.

Una vez ingresado el nuevo usuario en la base de datos, se redirige al usuario a la página principal. Donde al usuario se le pone también una descripción básica al principio y una foto de perfil predefinida.

```
async function storeUser(req, res) {
    const { email, nombre, contrasena } = req.body;

    try {
        //verificar si ya existe
        const existe = await req.db.query(
            'SELECT * FROM usuario WHERE email = $1',
            [email]
        );

        if (existe.rows.length > 0) {
            return res.render('vistas/usuario/registro', {
                title: 'Registro | Descubre tu alrededor',
                bodyClass: 'loginRegistro',
                mostrarNav: false,
                error: 'Ya existe un usuario con ese email'
            });
        }

        //encriptar contraseña
        const hash = await bcrypt.hash(contrasena, 12);

        //poner una foto de perfil automatica a un nuevo usuario
        const defaultAvatar = '/uploads/AvatarDefault.jpg';

        //insertar usuario
        await req.db.query(
            'INSERT INTO usuario (email, nombre, contrasena, rol,
descripcion, foto_perfil) VALUES ($1, $2, $3, $4, $5, $6)',
```

```
            [email, nombre, hash, 'Usuario', 'Hola soy un/a nuev@ usuario',
defaultAvatar]
        );

        //iniciar sesión al registrarse
        req.session.loggendin = true;
        req.session.email = email;
        req.session.nombre = nombre;
        req.session.rol = 'Usuario';
        req.session.descripcion = 'Hola soy un/a nuev@ usuario';
        req.session.foto_perfil = defaultAvatar;

        res.redirect('/');

    } catch (error) {
        console.error(error);
        res.send('Error en el registro');
    }
}
```

Inicio de sesión

Cuando el usuario carga la vista del login, se le muestra el formulario para poder iniciar sesión. Primero el html verifica ya si se ha ingresado bien todos los datos del formulario. Después de se hace transmiten los datos mediante una petición POST al controlador llamador LoginController.js

Antes de mandar al usuario a la vista principal se verifica una serie de datos. Primero si el usuario con el email ingresado no está en la base de datos sale un mensaje de error diciendo de que ese usuario con ese email no existe.

Luego si el usuario con el email ingresado existe, el controlador compara la contraseña que se ha encriptado con la contraseña encriptada en la base de datos. Si la contraseña ingresada no coincide, sale un mensaje de error diciendo de que el usuario se ha equivocado al ingresar la contraseña.

Pero si los datos que ha ingresado el usuario son completamente correctos, se cargara la vista principal con la sesión iniciada del usuario.

```
async function aut(req, res) {
    const { email, contrasena } = req.body;

    try {
        const result = await req.db.query(
            'SELECT * FROM Usuario WHERE Email = $1',
            [email]
        );

        if (result.rows.length === 0) {
            return res.render('vistas/usuario/inicioSesion', {
                title: 'Inicio sesión | Descubre tu alrededor',
```

```
                bodyClass: 'loginRegistro',
                mostrarNav: false,
                error: 'No existe un usuario con ese email'
            });
        }

        const usuario = result.rows[0];

        const passwordCorrecta = await bcrypt.compare(
            contrasena,
            usuario.contrasena
        );

        if (!passwordCorrecta) {
            return res.render('vistas/usuario/inicioSesion', {
                title: 'Inicio sesión | Descubre tu alrededor',
                bodyClass: 'loginRegistro',
                mostrarNav: false,
                error: 'Contraseña incorrecta'
            });
        }

        req.session.loggendin = true;
        req.session.email = usuario.email;
        req.session.contrasena = usuario.contrasena;
        req.session.nombre = usuario.nombre;
        req.session.rol = usuario.rol;
        req.session.descripcion = usuario.descripcion;
        req.session.foto_perfil = usuario.foto_perfil;

        res.redirect('/');

    } catch (error) {
        console.error(error);
        res.send('Error en el login');
    }
}
```

Añadir una ruta

Primero el usuario en la vista de ver las rutas, hay un botón arriba del todo para poder añadir una nueva ruta. Antes de cargar la vista con el formulario de añadir la nueva ruta, se verifica que el usuario este autenticado con el archivo de auth.js, que si el usuario no está autenticado se le cargara la vista del login.

Pero si el usuario esta autenticado, se podrá ver el formulario que está en la vista de añadir ruta. Ahí el usuario deberá de ingresar los datos de la ruta, nombre, dificultad, la descripción, el tiempo… Después de que el usuario, mande el formulario se realiza una petición POST, al archivo de rutasController.js. Donde ahí se va a verificar que no hay ninguna ruta con el mismo

nombre, si hay una ruta con el mismo nombre sale un mensaje de error, ya que ya existe una ruta con el mismo nombre.

Luego también se verifica, de que la ruta se haya hecho de manera correcta, eso se verifica que en el controlador de que haya más de 2 coordenadas ingresadas. Ya que los caminos que se muestran en el mapa que está en el formulario, para poder hacer las rutas van haciendo los caminos de la ruta con las coordenadas, por eso se verifica que haya más de 2 coordenadas. Además de eso hay el uso del api de nominantim, para obtener la dirección de la ruta.

```
async function anadirRuta(req, res) {
    const {
        nombre_ruta,
        dificultad_ruta,
        km,
        direccion,
        descripcion,
        desnivel_pos,
        desnivel_neg,
        altura_max,
        altura_min,
        duracion_estimacion,
        tipo_ruta,
        coordenadas
    } = req.body;

    const email = req.session.email;

    try {
        //las coordenadas
        const coords = JSON.parse(coordenadas);

        if (!coords || coords.length < 2) {
            return res.send("Ruta inválida");
        }

        const lineString = coords
            .map(p => `${p[1]} ${p[0]}`)
            .join(',');

        const geom = `LINESTRING(${lineString})`;

        const existe = await req.db.query(
            'SELECT 1 FROM ruta WHERE nombre_ruta = $1',
            [nombre_ruta]
        );

        if (existe.rows.length > 0) {
            return res.render('vistas/rutas/anadirRuta', {
                title: 'Añadir nueva ruta | Descubre tu alrededor',
                bodyClass: 'anadirRuta',
```

```
                mostrarNav: true,
                nombre: req.session.nombre,
                error: 'Ya existe una ruta con ese nombre'
            });
        }

        if (altura_min > altura_max) {
            return res.render('vistas/rutas/anadirRuta', {
                title: 'Añadir nueva ruta | Descubre tualrededor',
                bodyClass: 'anadirRuta',
                mostrarNav: true,
                nombre: req.session.nombre,
                error: 'La altura minima no puede superior a la altura
maxima'
            });
        }

        if (desnivel_pos < 0 || desnivel_neg < 0) {
            return res.render('vistas/rutas/anadirRuta', {
                title: 'Añadir nueva ruta | Descubre tualrededor',
                bodyClass: 'anadirRuta',
                mostrarNav: true,
                nombre: req.session.nombre,
                error: 'Los desniveles no pueden ser negativos'
            });
        }

        await req.db.query(
            `INSERT INTO ruta
            (nombre_ruta, dificultad_ruta, km, direccion, descripcion,
desnivel_pos, desnivel_neg, altura_max, altura_min, geom, email,
duracion_estimacion, tipo_ruta)
            VALUES
            ($1, $2, $3, $4, $5, $6, $7, $8, $9, ST_GeomFromText($10, 4326),
$11, $12, $13)`,
            [
                nombre_ruta,
                dificultad_ruta,
                km,
                direccion,
                descripcion,
                desnivel_pos,
                desnivel_neg,
                altura_max,
                altura_min,
                geom,
                email,
                duracion_estimacion,
                tipo_ruta
```

```
            ]
        );

        res.redirect('/verRutas');

    } catch (error) {
        console.error(error);
        res.status(500).send('Error al añadir una nueva ruta: ' +
error.message);
    }
}
```

Marcar ruta como hecha y comentar la ruta

Cuando el usuario ha cargado la vista del detalle de la ruta para poder marcar la ruta como hecha, primero el usuario debe de estar autenticado para poder marcar la ruta como hecha, si el usuario no está autenticado se le mandará a la vista de login, para que inicie sesión y luego podrá marcar la ruta como hecha.

Luego si el usuario esta autenticada que será lo normal, se hará una petición POST a su controlador para poder marcar la ruta como hecha. Primero se va a verificar que la ruta exista con el ID, y si la ruta existe pues se agrega la ruta al historial. Además de eso en la vista del detalle de la ruta, el usuario no podrá volver a marcar la ruta como hecha, ya que tengo puesto de que no aparezca al usuario de nuevo la opción para poder marcar la ruta como hecha.

Después el usuario, cuando haya marcado la ruta como hecha, le aparece un formulario para poder añadir un comentario. Y en el comentario lo que puede poner es el comentario con una imagen si el usuario lo desea, aunque no esté obligado a poner la imagen. Cuando el usuario manda el comentario, se hace una petición POST a archivo de comentarioController.js.

```
async function anadirComentario(req, res) {
    const { comentario } = req.body;
    const email = req.session.email;
    const { id } = req.params;

    try {
        if (!email) {
            return res.status(401).send("Usuario no autenticado");
        }

        const rutaHecha = await req.db.query(
            'SELECT * FROM historial_rutas WHERE email = $1 AND id_ruta =
$2',
            [email, id]
        );

        if (rutaHecha.rows.length === 0) {
            return res.status(404).send("No puedes comentar esta ruta si no
la has hecho");
        }
```

```
        const id_historial = rutaHecha.rows[0].id_historial;

        let url_imagen = null;
        if (req.file) {
            url_imagen = '/fotosComentarios/' + req.file.filename;
        }

        await req.db.query(
            "INSERT INTO comentario_ruta (id_historial, comentario,
url_imagen) VALUES ($1, $2, $3)",
            [
                id_historial,
                comentario,
                url_imagen
            ]
        );

        const id_ruta = rutaHecha.rows[0].id_ruta;
        res.redirect(`/detalleRuta/${id_ruta}`);
    } catch (error) {
        console.error(error);
        res.status(500).send('Error al añadir el comentario: ' +
error.message);
    }
}
```

Editar ruta

Primero cuando el usuario debe de estar en la vista del detalle de la ruta, si el usuario que ha creado la ruta está viendo esa vista tendrá arriba la opción de editar ruta. Cuando el usuario entre en la vista de editar ruta, se verá un formulario con los datos de la ruta para modificarlos. Se verán todos menos los km, el mapa de la ruta, altitud máxima y mínima, y el desnivel positivo y negativo.

El resto de los datos se pueden modificar, luego cuando se envía el formulario se hace una petición POST al controlador que es rutaController.js, para poder modificar los datos de la ruta y se actualizarán de manera correcta en la base de datos. Antes de subir la ruta se verifica de manera correcta de que el usuario que vaya a modificar los datos de la ruta es bien el usuario que haya creado la ruta.

```
async function modificarRuta(req, res) {
    const {
        nombre_ruta,
        dificultad_ruta,
        descripcion,
        duracion_estimacion,
        tipo_ruta
    } = req.body;

    const email = req.session.email;
```

```
    const { id } = req.params;

    try {
        if (!email) {
            return res.status(401).send("Sesión no válida");
        }

        const result = await req.db.query(`
            UPDATE ruta
            SET
                nombre_ruta = $1,
                dificultad_ruta = $2,
                descripcion = $3,
                duracion_estimacion = $4,
                tipo_ruta = $5
            WHERE id_ruta = $6 AND email = $7
        `, [
            nombre_ruta,
            dificultad_ruta,
            descripcion,
            duracion_estimacion,
            tipo_ruta,
            id,
            email
        ]);

        if (result.rowCount === 0) {
            return res.status(403).send("No tienes permisos para modificar
esta ruta");
        }

        res.redirect(`/detalleRuta/${id}`);

    } catch (error) {
        console.error(error);
        res.status(500).send('Error al modificar la ruta: ' + error.message);
    }
}
```

Organizar quedada

Primero el usuario debe de estar en la vista del detalle de ruta, en esa misma vista más abajo hay un botón elegido para poder organizar la quedada. Primero cuando el usuario va a organizar la quedada se verifica que el usuario esta autenticado correctamente, si no está autenticado se le mandara a la vista del login, para que se autentique y pueda organizar la quedada.

Si el usuario esta autenticado, lo que se hará luego será verificar que la ruta que ha elegido el usuario para hacer la quedada existe de forma correcta. Después de eso, se le mostrara al usuario el formulario para poder organizar la quedada, donde ingresara el día, la dirección, una

descripción si lo desea el usuario y el máximo de participantes. Luego se hace una petición POST al archivo de quedadaController.js con todos los datos del formulario con su controlador correspondiente y la quedada se subirá a la base de datos, para que los usuarios puedan apuntarse a la quedada.

```
async function organizarQuedada(req, res) {
    const {
        dia,
        hora,
        direcion,
        descripcion,
        max_participantes
    } = req.body;

    const email = req.session.email;
    const { id } = req.params;

    try {
        if (!email) {
            return res.status(401).send("No autenticado");
        }

        const resultadoRuta = await req.db.query(
            'SELECT email FROM ruta WHERE id_ruta = $1',
            [id]
        );

        if (resultadoRuta.rows.length === 0) {
            return res.status(404).send("Ruta no encontrada");
        }

        await req.db.query(
            'INSERT INTO quedada (id_ruta, email_creador, dia, hora,
direcion, descripcion, max_participantes) VALUES ($1, $2, $3, $4, $5, $6,
$7)',
            [
                id,
                email,
                dia,
                hora,
                direcion,
                descripcion,
                max_participantes
            ]
        );

        res.redirect(`/detalleRuta/${id}`);
    } catch (error) {
        console.error(error);
```

```
        res.status(500).send('Error al añadir una nueva ruta: ' +
error.message);
    }
}
```

Ver detalle de la quedada

Primero para poder ver el detalle de la quedada el usuario, da igual que este autenticado o no que podrá llegar a ver el detalle de la quedada. Primero cuando el usuario selecciona la quedada de una ruta se hace una petición GET al controlador con el nombre de quedadaController.js, y luego se verá el detalle de la quedada de la ruta elegida.

Luego se cargará la vista del detalle de la quedada con todos los datos de la quedada, el nombre, datos importantes, descripción de la ruta. También se va a ver la forma de la ruta que se ha hecho para esa ruta. Donde la forma se pilla de un array que hay en la base de datos con todas las coordenadas que ha ido por puntos en la ruta. Y se genera con eso el mapa de la ruta para la vista del usuario que este viendo el detalle de la quedada.

Luego también se juntó la tabla de la ruta con la tabla de fotos de la ruta, donde el usuario que ha añadido la ruta se podrán ver las imágenes que el usuario ha añadido. Luego también se junta con la tabla de comentario, para que los usuarios que hayan hecho la ruta puedan compartir su propia experiencia de la ruta, con un comentario y una imagen si desean añadir.

```
async function detalleQuedada(req, res) {
    try {
        const id = parseInt(req.params.id);

        if (isNaN(id)) {
            return res.send("ID no valido");
        }

        const resultadoQuedada = await req.db.query(`
            SELECT q.*, r.nombre_ruta, u.nombre, ST_AsGeoJSON(geom) as
geom_json
            FROM quedada q
            JOIN ruta r ON q.id_ruta = r.id_ruta
            JOIN usuario u ON q.email_creador = u.email
            WHERE q.id_quedada = $1
        `, [id]);

        if (resultadoQuedada.rows.length === 0) {
            return res.status(404).send("Quedada no encontrada");
        }

        const quedada = resultadoQuedada.rows[0];
        const emailUsuario = req.session.email || null;

        let esApuntado = false;
        if (emailUsuario) {
            const resultadoParticipante = await req.db.query(
```

```
                'SELECT 1 FROM participantes_quedada WHERE id_quedada = $1
AND email = $2',
                [id, emailUsuario]
            );
            esApuntado = resultadoParticipante.rows.length > 0;
        }

        res.render('vistas/quedada/detalleQuedada', {
            title: 'Quedada de ' + quedada.nombre_ruta + " | Descubre tu
alrededor",
            bodyClass: 'detalleQuedada',
            mostrarNav: true,
            nombre: req.session.nombre,
            quedada: quedada,
            esApuntado: esApuntado,
            emailUsuario: emailUsuario
        });
    } catch (error) {
        console.error(error);
        res.status(500).send('Error al obtener las rutas: ' + error.message);
    }
}
```

Reportar ruta

Para poder reportar una ruta, primero el usuario debe de estar autenticado, pero si el usuario no está autenticado, le mandan a la vista del login, para que tenga que iniciar sesión y pueda reportar la ruta. Luego si el usuario ha iniciado la sesión, el podrá reportar la ruta.

Cuando el usuario selecciona la opción de reportar la ruta, va a tener que rellenar un formulario donde tendrá que ingresar el reporte de la ruta. Cuando el usuario mande el formulario, se hará una petición POST al controlador reporteRuta.js, para poder ingresar el reporte en la base de datos.

```
async function hacerReporte(req, res) {
    const { texto_reporte } = req.body;

    const email = req.session.email;
    const { id } = req.params;

    try {
        if (!email) {
            return res.status(401).send("Sesión no válida");
        }

        const result = await req.db.query(`
            INSERT INTO reporte (email, id_ruta, texto_reporte) VALUES ($1,
$2, $3)`,
            [
                email,
                id,
```

```
                texto_reporte
            ]);

        res.redirect(`/detalleRuta/${id}`);
    } catch (error) {
        console.error(error);
        res.status(500).send('Error al añadir el reporte de la ruta: ' +
error.message);
    }
}
```

Mapa interactivo

Primero cuando el usuario carga la vista de vistaGPS.hbs, se le mostrara un mapa interactivo donde el usuario va a poder moverse, y podrá ver varios puntos. Cada uno de esos puntos servirán al usuario para poder ver donde hay una ruta. Cuando el usuario hace clic en uno de esos puntos, se le mostrara el nombre de la ruta y además de eso se le va a mostrar también una opción para poder ver el detalle de esa ruta.

En el mapa interactivo se hace una petición GET hacia el controlador con nombre rutasGPSController.js.

```
async function verGPS(req, res) {
    try {
        const result = await req.db.query(
            `SELECT id_ruta, nombre_ruta, ST_AsGeoJSON(geom) AS geom_json
            FROM ruta`
        );
        const routes = result.rows;

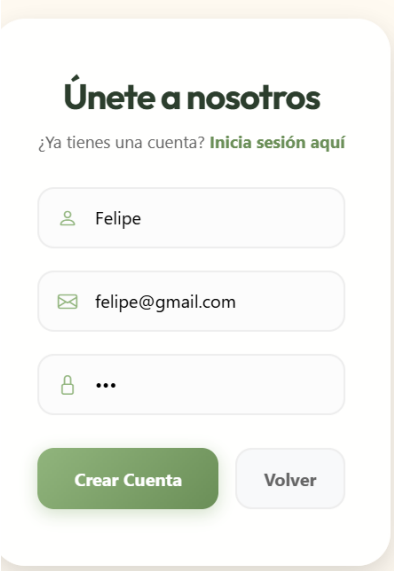
        res.render('vistas/gps/vistaGPS', {
            title: 'Ver rutas | Descubre tualrededor',
            bodyClass: 'verRutas',
            mostrarNav: true,
            nombre: req.session.nombre,
            routes: routes
        })
    } catch (error) {
        console.error(error);
        res.status(500).send('Error al ver la vista GPS: ' + error.message);
    }
}
```

Resultados y análisis

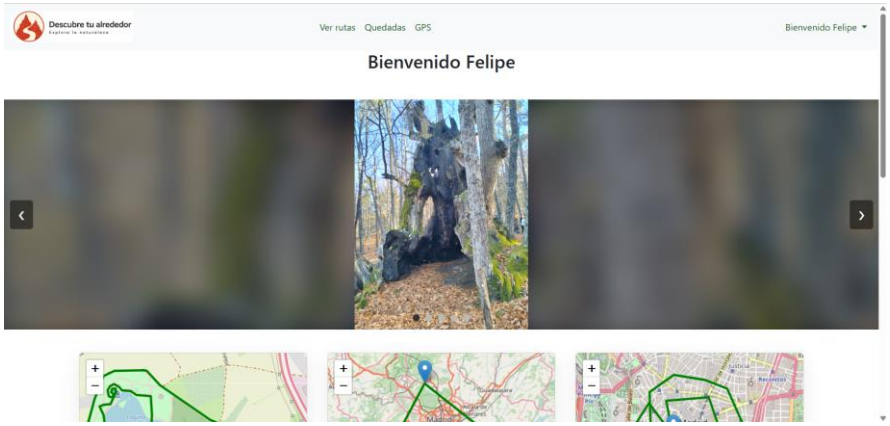
Resultados

Descripción detallada de los resultados obtenidos. Se pueden aportar recortes de pantalla que ilustran el funcionamiento del programa en ejecución

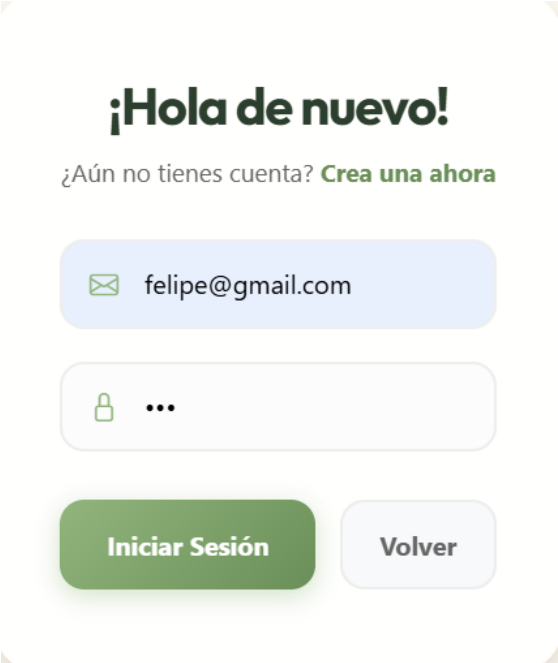
Registro



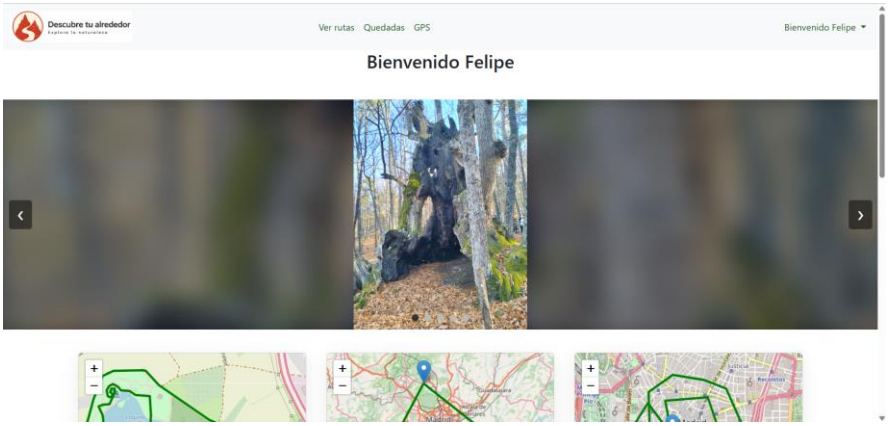
Aquí el usuario que quiera registrarse tiene que rellenar el formulario con sus datos, el nombre, su email y su contraseña. Y cuando le da clic a “crear cuenta”, le mandara a la vista index.hbs



Iniciar sesión



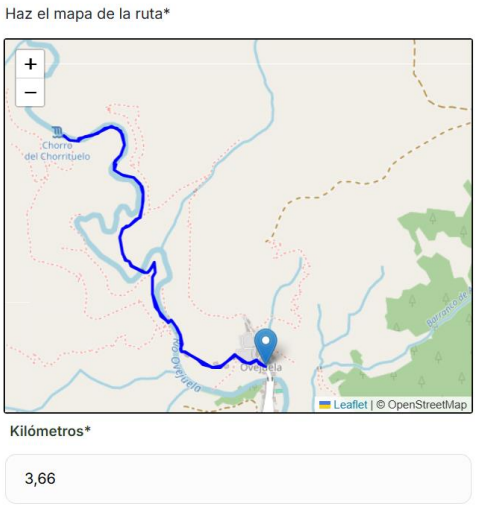
Aquí está la vista del formulario para poder iniciar sesión. Donde el usuario tendrá que poner el email y su contraseña y luego tendrá que hacer clic en “Iniciar Sesión”, y le mandará al usuario a la vista de index.hbs



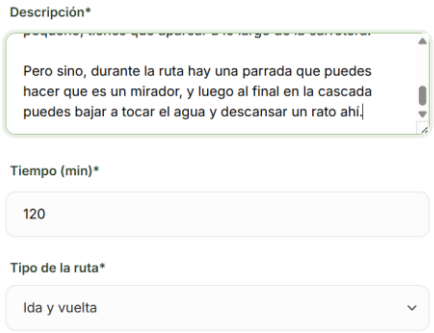
Añadir una ruta



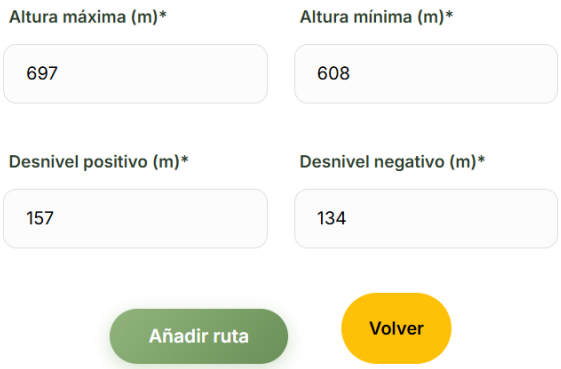
Primero el usuario debe de rellenar un formulario, primero pondrá el nombre de la ruta y la dificultad



Luego el usuario debe de hacer la ruta con el api de leaflet, donde debe de hacer la ruta. La ruta se hace mediante a que el usuario va haciendo clic en el mapa y crea un punto, cada uno de esos puntos es una coordenada. Y se van juntando los puntos para poder hacer la ruta, además de eso los km se ponen automáticamente ya que calcula la distancia



Aquí el usuario, debe poner una descripción de la ruta puede ser corta o larga da igual de como es, y es recomendable poner aquí la descripción de la ruta. Luego tiene que poner el tiempo en minutos, y luego poner el tipo de ruta si es ida y vuelta que se va a usar para las rutas que se vuelve por el mismo recorrido. Y si la ruta es circular significa que esta ruta se va por un camino y se vuelve por otro diferente.



Aquí el usuario debe de poner la altura máxima y mínima y el desnivel positivo y negativo que hay.

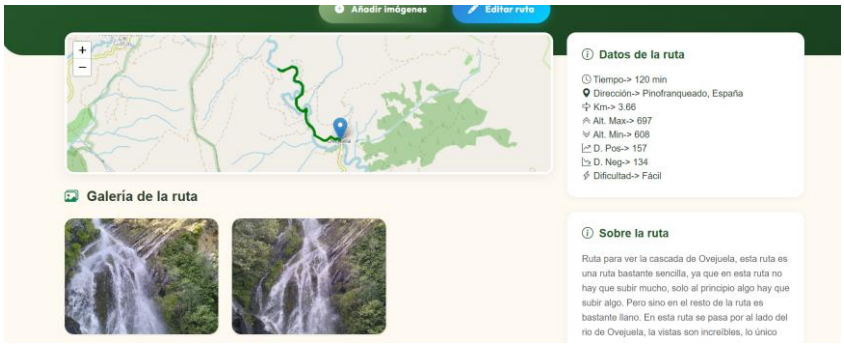
Añadir imágenes ruta



Al que el usuario “Felipe” ha creado la ruta de Ovejuela, tiene dos opciones que le aparecen al principio, la que vamos a ver ahora es la opción de añadir imágenes. Cuando el usuario hace clic en la opción de añadir imagen, le mandara a un formulario donde podrá añadir imágenes de la ruta, para que la gente vea lo bonita que es.

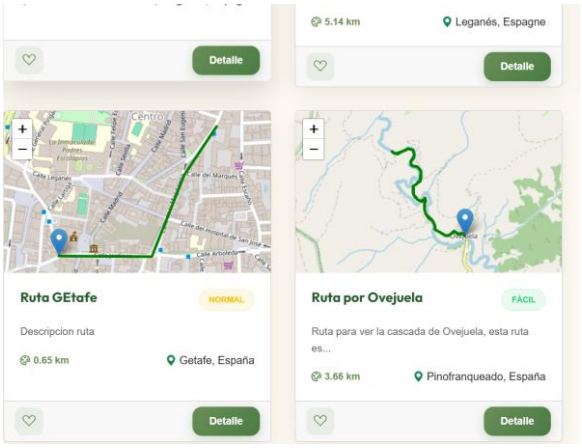


Aquí esta el formulario para añadir las imágenes de la ruta, cuando el usuario sube unas imágenes con un limite de 6. Cuando las tiene elegidas el usuario debe de hacer clic en “Subir imágenes”

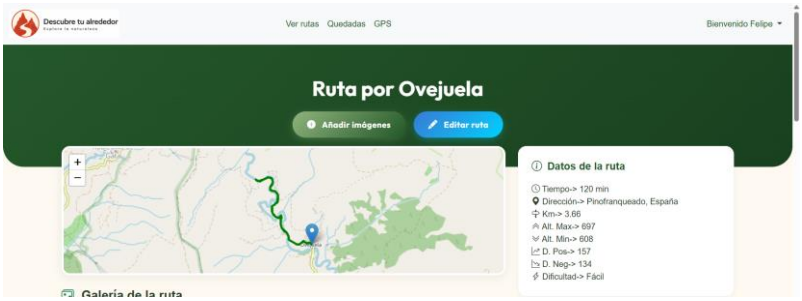


Cuando el usuario sube las nuevas imágenes, se verán las imágenes que ha subido el usuario en la galería de la ruta. El usuario puede añadir aún más imágenes de todas formas, si el desea.

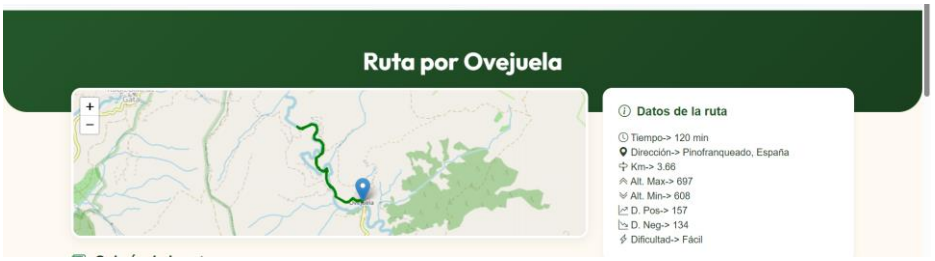
Detalle ruta



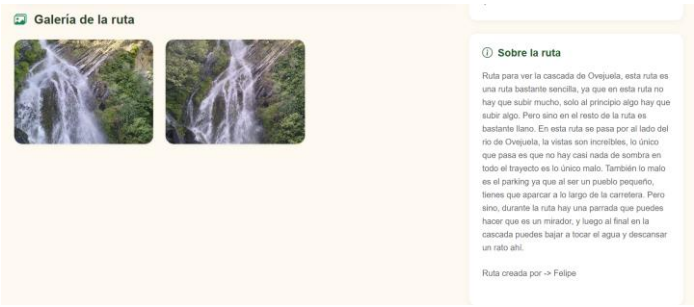
En la vista de verRutas.hbs, es donde esta el listado de todas las rutas. Y en cada ruta como se puede ver hay algo de información sobre ellas, nombre de la ruta, la dificultad, el mapa de la ruta, la descripción, pero si es muy larga la acortan un poco, los km y la distancia. Además de eso, tiene la opción para poder la ruta en favoritos y en la de detalle ruta. Este botón es lo que vamos a ver ahora, este botón servirá para poder ver el detalle de la ruta. Cuando el usuario hace clic le manda al usuario a la vista detalleRuta.hbs



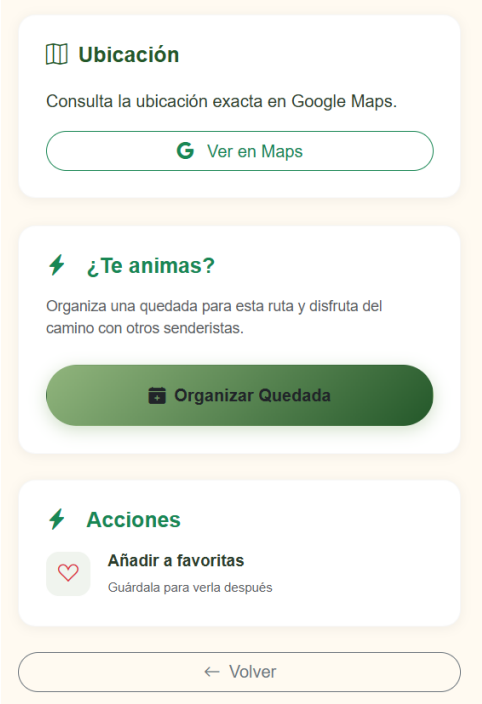
En la vista de detalleRuta.hbs, se va a ver arriba del todo el nombre de la ruta y dos botones que solo le aparecen a los usuarios que han creado la ruta, esos botones manda al usuario a un formulario, uno es para añadir nuevas imágenes a la ruta, y la otra sirve para poder editar los datos de la ruta. Luego un poco más abajo está el mapa de la ruta, y al lado esta unos datos de la ruta como el tiempo que se tarda, los km, la altura máxima y mínima, el desnivel positivo y negativo y la dificultad de la ruta.



Cuando el usuario que esta viendo el detalle de la ruta, no es el creador de la ruta se vera de esta forma, sin los 2 botones de añadir imagen y la de editar ruta.



Luego un poco mas abajo, hay una galería de imágenes que el creador de la ruta ha añadido el mismo, pero si no hay un mensaje donde pone que hay que esperar a que el usuario que ha hecho la ruta agregue imágenes al detalle de la ruta. Luego en la derecha se puede ver la descripción de la ruta, y que usuario ha creado la ruta.



Luego aquí hay varias opciones, la primera es obtener la dirección en donde empieza la ruta mediante las coordenadas, cuando el usuario hace clic en la opción de Google maps mandara al usuario a Google maps con la dirección en donde empieza la ruta.

Después esta la opción de organizar quedada mandara al usuario a un formulario para que el usuario organiza una quedada.

Y luego el ultimo botón, en esta vista al estar con el usuario que ha hecho la ruta se vera solo una opción que es la de añadir la ruta a favoritos, para poder guardar la ruta si se quiere hacer en un futuro.

Y luego está la opción de volver a la página anterior

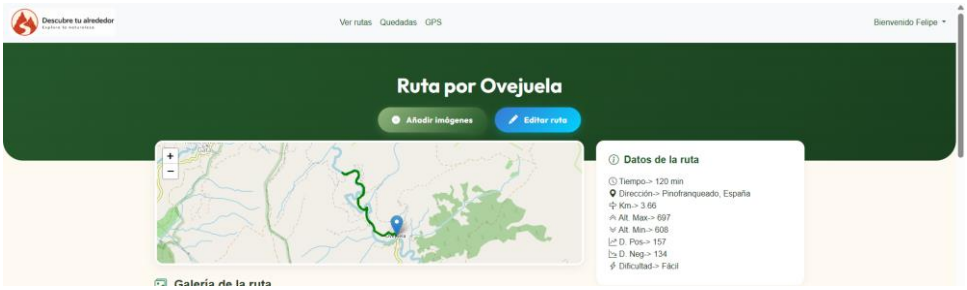


Si el usuario que esta viendo el detalle de la ruta, es uno que no ha creado la ruta, tiene la opción para marcar la ruta como completada, que sirve para poder añadir un comentario en la ruta. Y la opción para reportar la ruta, en donde manda al usuario a un formulario para poder escribir el reporte de la ruta.

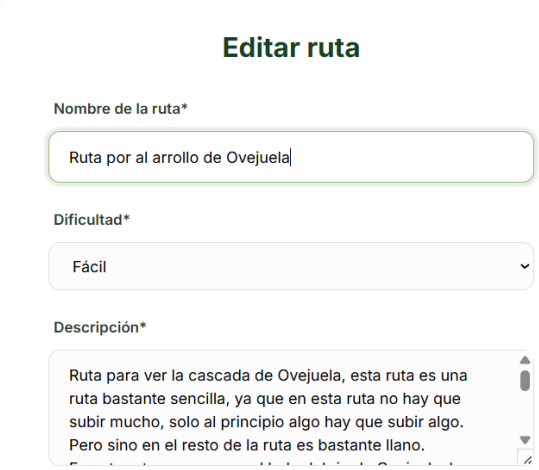


Aquí se puede ver la caja de comentarios de la ruta, con todos los comentarios de la ruta.

Editar ruta



Aquí esta opción esta disponible solo para el usuario que haya creado la ruta. Cuando se hace clic en “Editar ruta”, mandara al usuario a un formulario con todos los datos rellenados, que son los datos de la ruta. Y el usuario no deberá de rellenar por el mismo todos los datos y solo cambiará los datos que él quiera.



Aquí, están los datos ya rellenados, pero el usuario puede cambiar por ejemplo el nombre de la ruta, la dificultad o la descripción. Aquí en mi caso he modificado el nombre de la ruta para que sea más exacto.



Además de esos datos se puede modificar también el tiempo y el tipo de ruta.

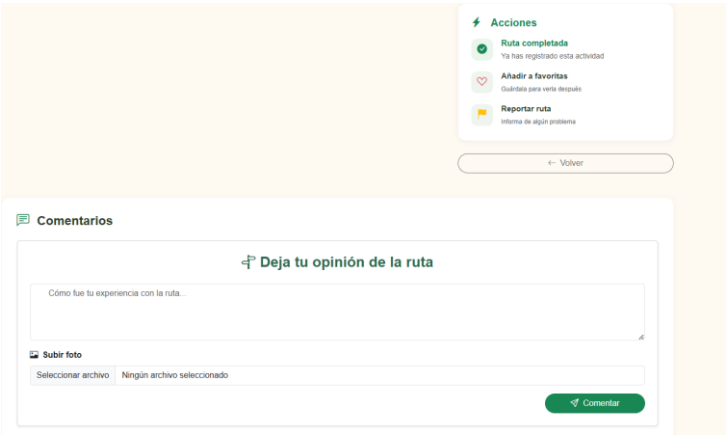


Aquí se ve que el nombre de la ruta se ha modificado de forma correcta

Marcar ruta como completa y añadir comentario



El usuario para poder marcar la ruta como hecha y poder comentar debe de ser un usuario que no haya hecho esa ruta. Primero el usuario debe de hacer clic en la opción de “Marcar ruta como hecha”, esa opción esta abajo antes de la sección de comentarios.



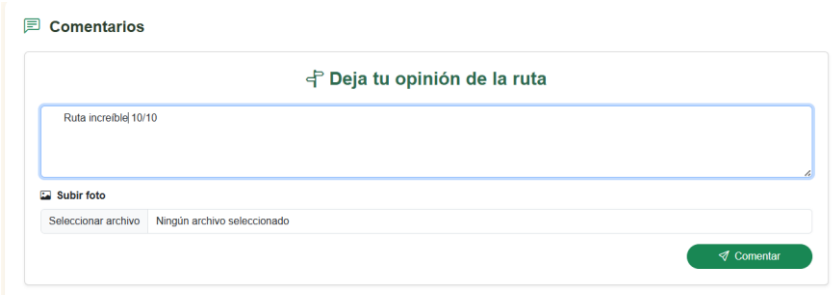
Una vez que el usuario haga clic en “Marcar ruta como hecha”, como se puede ver se marca la ruta como hecha y luego se abre un formulario abajo para poder dejar un comentario. El usuario puede poner un comentario con una imagen o no, para que el usuario pueda decir lo que opina de la ruta



Aquí hay un ejemplo de lo que el usuario puede poner como comentario.



Debajo del formulario se puede ver los comentarios dejados por los usuarios. Como se puede observar se ve la foto de perfil del usuario, el nombre de usuario y en qué fecha se subió el comentario. Además de eso se puede apreciar el comentario puesto por el usuario y también la foto que ha subido.

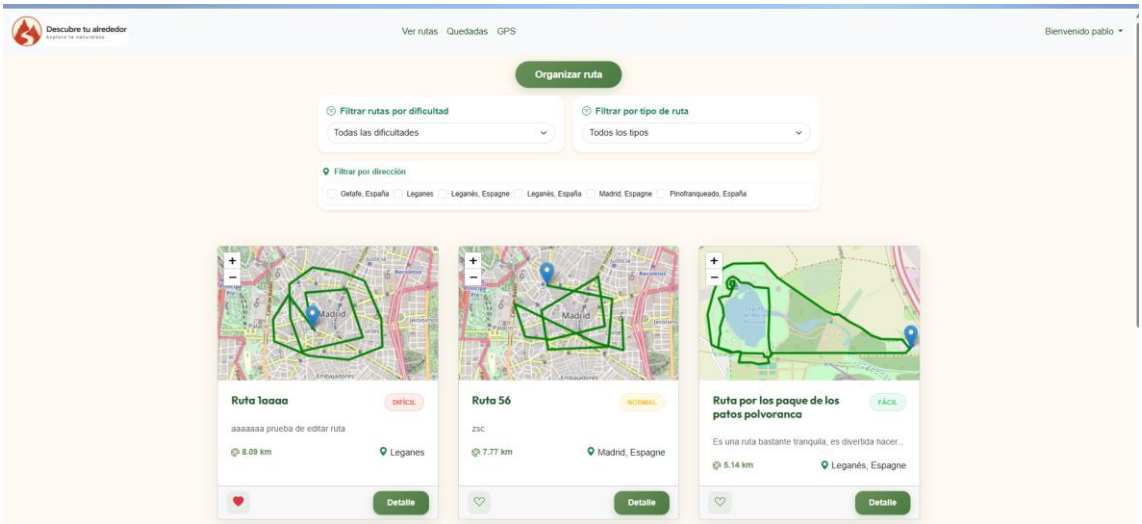


Este es un ejemplo de un comentario subido pero sin ninguna foto



Aquí esta el resultado de la vista, pero con el comentario sin imagen, es la que esta abajo del todo.

Ver rutas

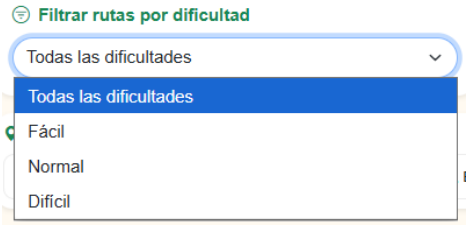


Aquí esta la vista de verRutas.hbs, en esta vista se podrán ver todas las rutas disponibles y luego el detalle de una en concreto que pueda ser que te atraiga bastante.

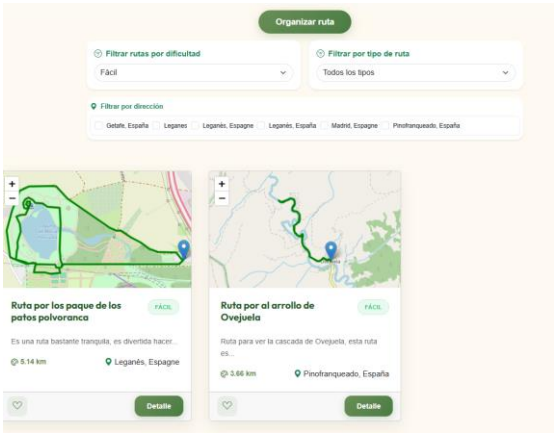
Además de eso, hay una opción para poder añadir una ruta que manda al usuario a un formulario para añadir las rutas.

Y también tenemos a nuestra disposición 3 filtros:

- Filtrar por dificultad

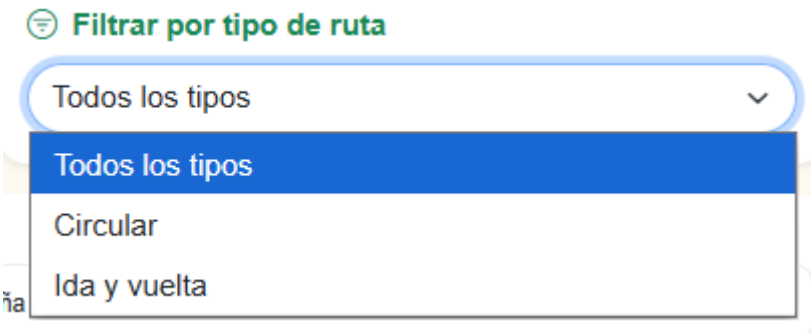
  

  Aquí se puede ver los 3 tipos de dificultades

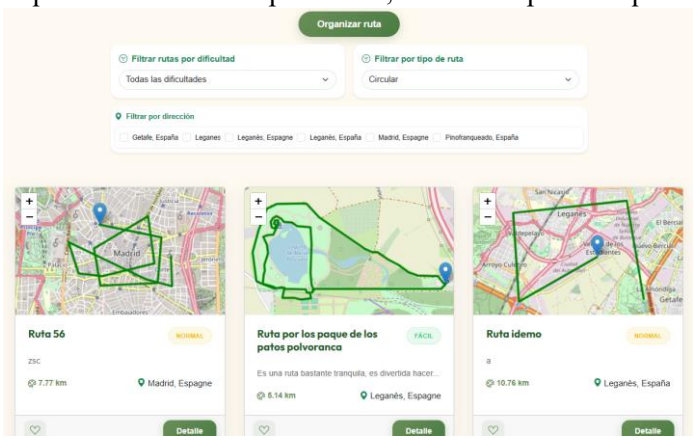


Aquí un ejemplo de cómo funciona ese filtro

- Filtrar por tipo de ruta

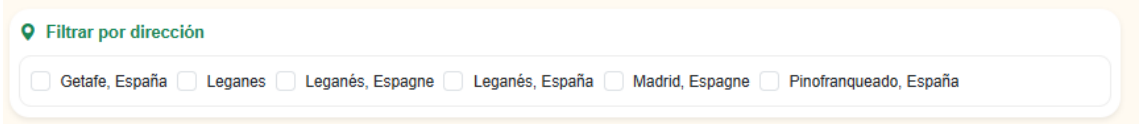


Aquí esta el filtro de tipo de ruta, con las 2 opciones que tiene

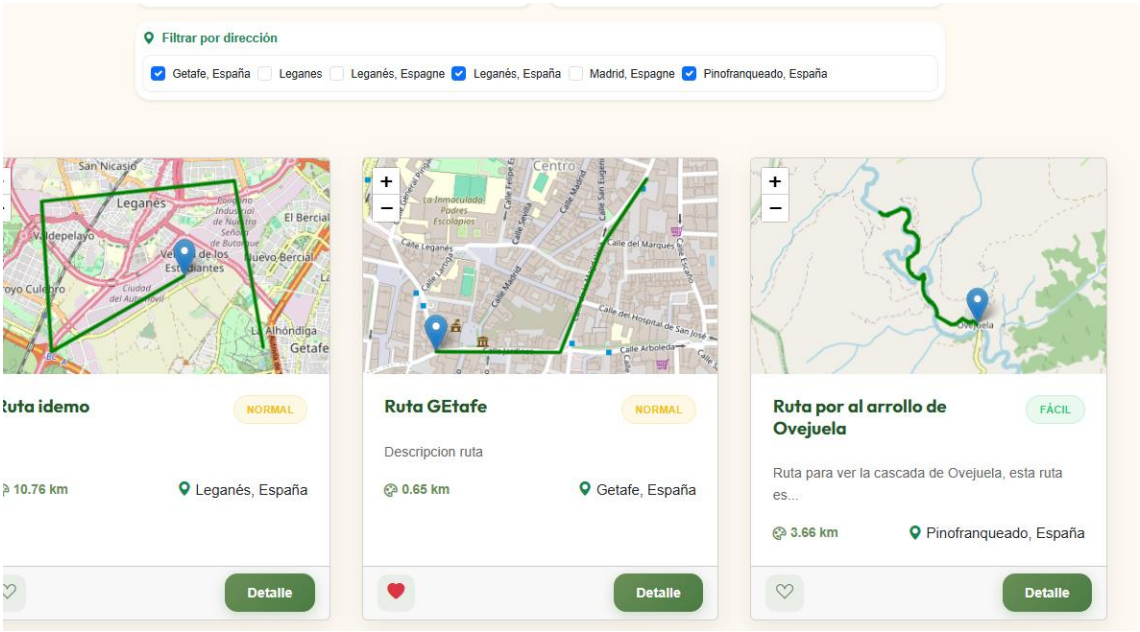


Ejemplo de cómo funciona el filtro

- Filtrar por dirección

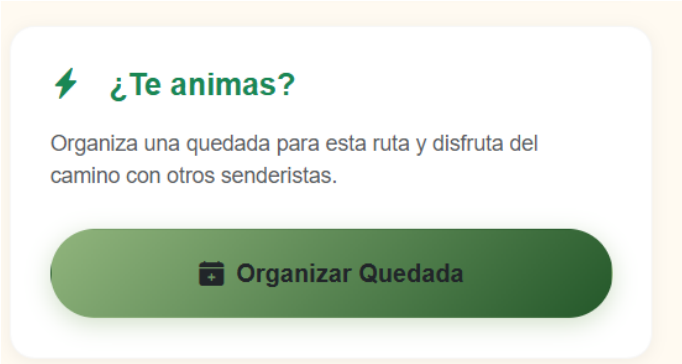


Este filtro, es una checkbox, para poder pillar las direcciones de varias rutas

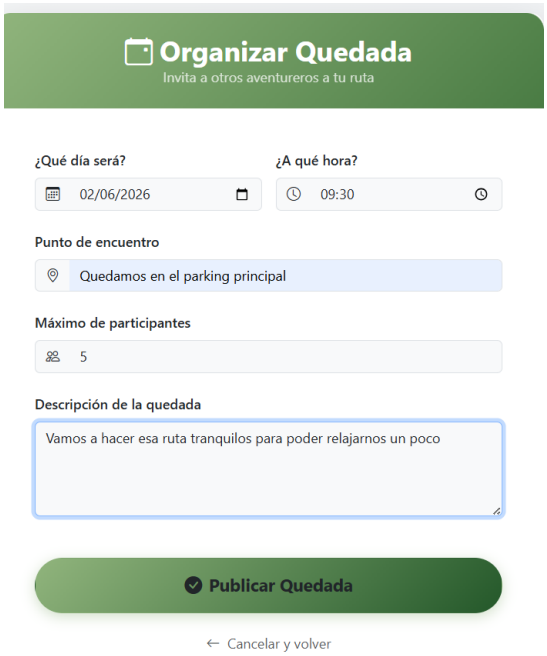


Aquí esta un ejemplo, el usuario ha seleccionado 3 direcciones diferentes y se han imprimido las rutas que hay en esas 3 dirección diferente.

Organizar quedada

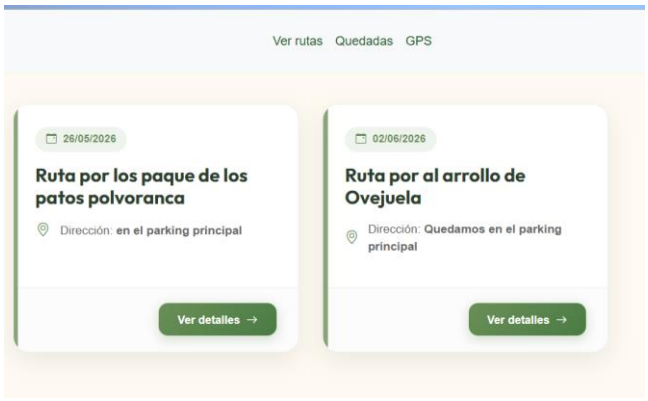


Primero para poder organizar la quedada el usuario, tiene que dirigirse al detalle de la quedada. Y luego bajar hasta la parte, donde hay un botón que va a permitir al usuario poder organizar la ruta



Cuando el usuario hace clic en la opción para organizar la quedada, le manda a un formulario que tiene que completar. Tiene que poner la fecha de la ruta, la hora, el punto de encuentro, máximo de participantes de la ruta y una descripción, aquí se podría poner lo que se va a hacer durante la quedada. Y después se hace clic en “Publicar Quedada”.

Apuntarse a quedada

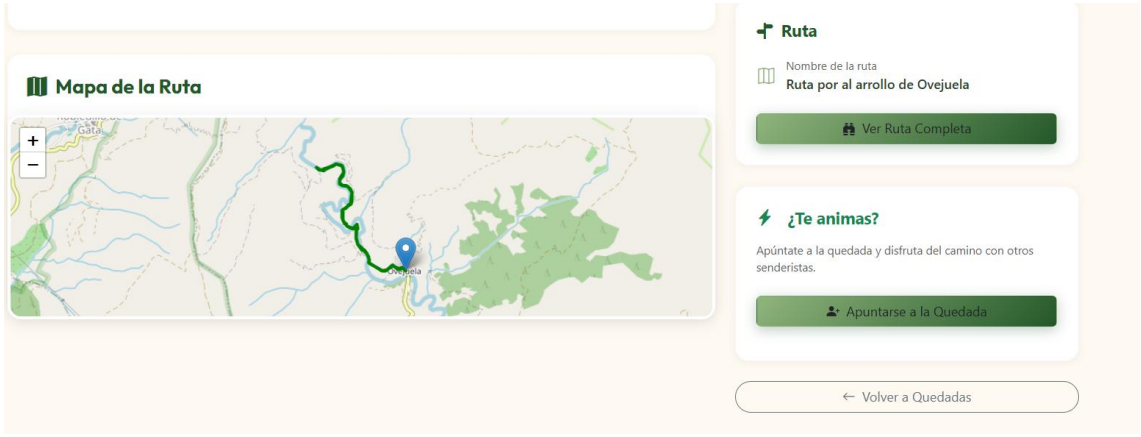


Para poder apuntarse a una quedada, primero solo los usuarios que no han creado la quedada se pueden apuntar. Después de eso, para poder ver las quedadas disponibles, hay que irse a la vista de verQuedadas.hbs, donde habrá un catálogo de las próximas quedadas, solo aparecerán las quedadas próximas. Por ejemplo, si una quedada ha sido antes del día actual ya no aparecerá el listado de esa quedada.

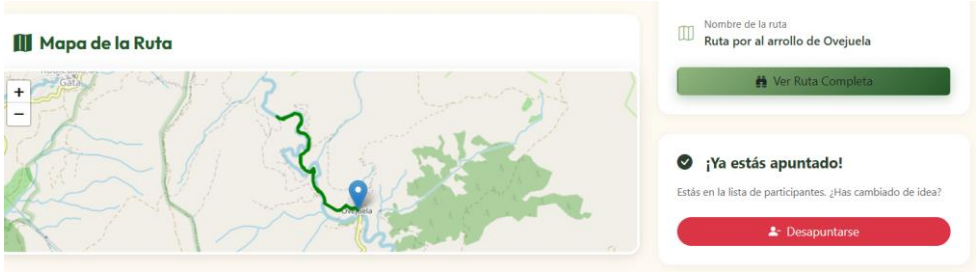
Para poder apuntarse a una quedada, tiene que hacer clic en la opción de “Ver detalles”



Al estar en el detalle de la quedada, se vera en que ruta es la quedada, con una descripción de ella, el punto de encuentro. Y unos datos adicionales, como la fecha, la hora, el máximo de participantes y quien ha sido que la ha organizado dicha quedada.

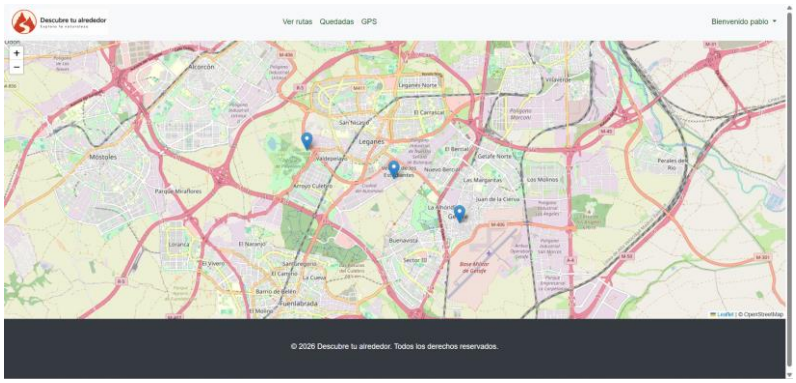


Mas abajo, se puede ver el mapa de la ruta, poder ver el detalle de la ruta que se va a realizar para saber que se les espera de la ruta. Y también está la opción para poder apuntarse a la quedada



Cuando el usuario se apunta a la quedada, como se puede ver la opción para apuntarse cambia a una nueva opción que es desapuntarse de la quedada.

Vista GPS



Esta es la vista GPS, donde el centro de la vista será en tu ubicación actual por coordenadas, pero si el usuario no tiene activo la ubicación se le pondrá en el centro de Madrid. Además de eso este mapa sirve para sabes que rutas tienes en tu zona para poder hacer.



Al hacer clic en un punto, se podrá ver el nombre de la ruta, y un enlace para poder ver el detalle de la ruta. Para que los usuarios puedan saber lo que les espera en la ruta.

Reportar ruta



Primero el usuario debe de entrar en el detalle de una ruta, además de eso el usuario que este viendo ese detalle de la ruta. Después de eso el usuario debe de bajar hasta la parte de “Acciones”, y debe de hacer clic en la parte de “Reportar ruta”.

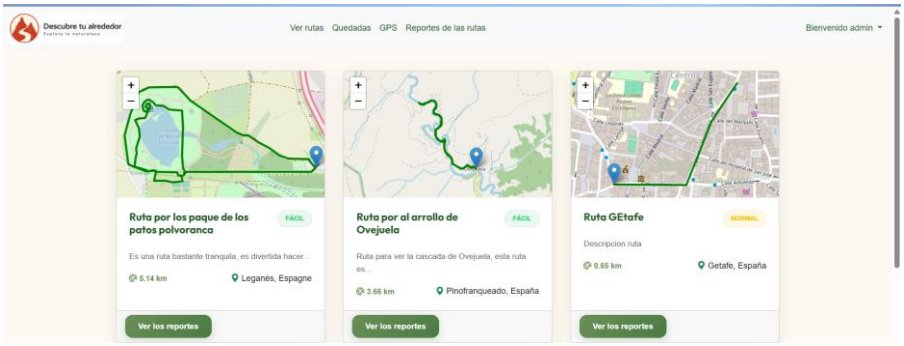


Aquí mandará al usuario a un formulario donde deberá de ingresar el reporte de la ruta. Y luego al enviar el reporte manda al usuario a la vista de detalle ruta.

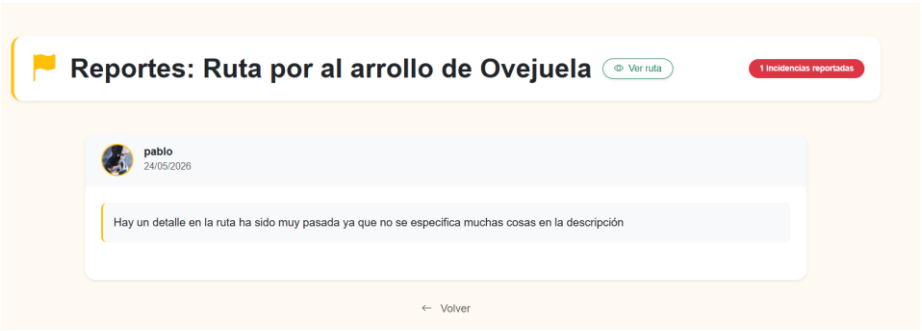
Ver reportes ruta (para vista admin)



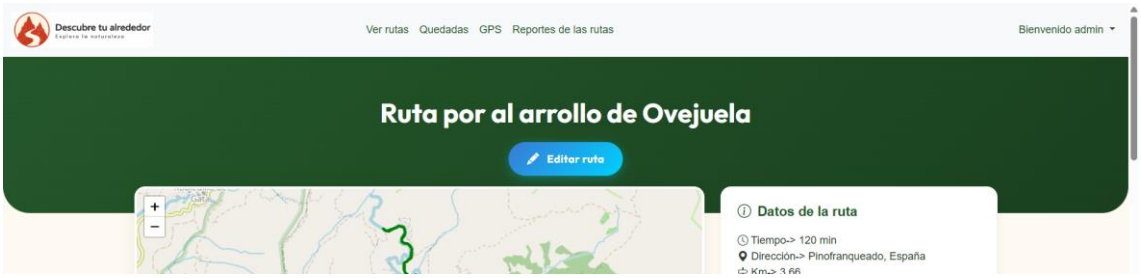
Para que los usuarios puedan ver los reportes, el usuario en cuestión debe de tener el rol de "Admin". Cuando un usuario se conecta, le saldrá una nueva opción llamada "Reportes de las rutas".



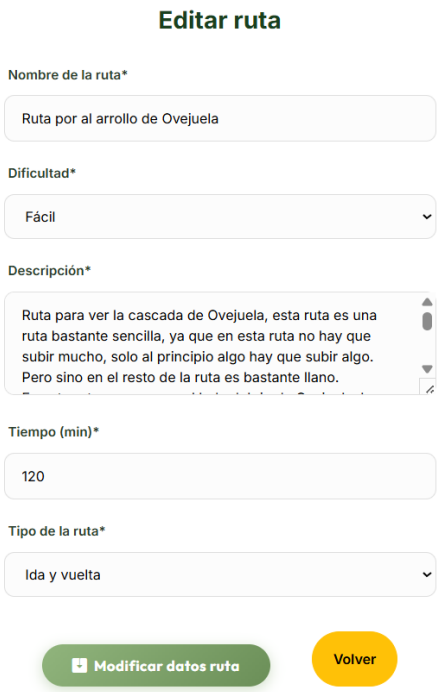
Cuando el usuario, está en la opción para poder ver los reportes, se le mostrar un listado con todas las rutas que tienen reportes.



Cuando se hace clic en "Ver los reportes", se le mostrara al usuario que reportes tiene tal ruta. Además, de eso, el total de reportes que hay, una opción para poder ver de detalle de la ruta. Y un poco más abajo todos los comentarios de reportes que han tenido la ruta.



Cuando el usuario con el rol de "Admin", le saldrá a él la opción de poder editar la ruta para poder modificar los datos que están mal.



Como se puede ver, al usuario con el rol de admin puede modificar los datos de la ruta.

Funcionalidades desarrolladas

Las funcionalidades que he desarrollado durante este proyecto:

- **Registro**
  - Lo que hace el usuario, es abrir la vista de registro y completa los datos del formulario, email, nombre y contraseña
- **Login**
  - Lo que hace aquí el usuario es abrir la vista de login y rellenar el formulario con su nombre y contraseña. Para poder añadir rutas, organizar quedadas y etc…
- **Añadir ruta**
  - El usuario rellena un formulario con los datos de la ruta, y luego hace el trazo del mapa
- **Ver rutas**
  - Aqui el usuario tiene a disponibilidad todas las rutas disponibles
- **Detalle de la ruta**
  - Servirá a los usuarios poder ver toda la información de la ruta
- **Editar ruta**
  - El usuario que haya creado la ruta puede modificar los datos

- **Añadir imagen**
  - El usuario que ha creado la ruta solo puede añadir imágenes de la ruta
- **Marcar ruta como hecha y añadir comentario**
  - Aqui el usuario debe de marcar la ruta como hecha, y podrá compartir su experiencia de la ruta
- **Ver dirección en maps**
  - El usuario debe de ir en detalle de la vista, y ver la dirección de la ruta, donde le mandara a donde empieza la ruta
- **Añadir la ruta a favorita**
  - Aquí el usuario puede añadir como favorita la ruta, en la zona donde están todas las rutas disponibles y en el detalle de la ruta.
- **Reportar ruta**
  - Aquí el usuario que no haya creado la ruta que esta viendo puede reportar la ruta, y tendrá que completar el formulario para hacer el reporte
- **Organizar una quedada**
  - Para organizar la quedada, hay que estar en el detalle de una ruta, y luego tiene que rellenar un formulario para poder organizar la quedada
- **Apuntarse a la quedada**
  - El usuario, debe de ir a la vista donde están todas las quedadas, y luego apuntarse a una de ellas
- **Vista GPS**
  - La vista gps sirve para poder ver las rutas que tienes por tu zona, y además de eso centra el usuario en su ubicación por coordenadas
- **Ver reportes y tomar acciones**
  - El usuario con el rol de admin, puede ir a la opción de ver reportes, donde le saldrá un listado con todas las rutas con reportes. Y podrá ver todos los reportes que tiene cada ruta. Además de eso admin puede modificar los datos de las rutas, por si hay errores.
- **Perfil del usuario**
  - También todos los usuarios tienen un perfil, donde pueden ver las rutas creadas por ellos mismos, el historial de rutas hechas y las rutas en favoritas. También ver las quedadas organizadas y en las que ha participado. Además, los usuarios pueden modificar su foto de perfil, contraseña y nombre de usuario con su descripción si ellos desean

Problemas encontrados durante el desarrollo y cómo se resolvieron

Los errores que me han salido durante el desarrollo han sido los siguientes

- Uno de los errores que me han surgido durante el desarrollo ha sido el siguiente. Cuando el usuario con el rol de admin cargaba la vista para poder editar los datos de la ruta, tenia puesto que buscara la ruta concreta con el id de la ruta. Y se me olvido quitar que pillara un email, y luego lo quite
- Otro error que tuve durante el desarrollo fue un error en el formulario del cambio de contraseña. El error que tenia era el siguiente, cuando el usuario tenia que ingresar 2 veces la misma contraseña para poder cambiarla. Pero si el usuario ingresaba la contraseña 2 veces, pero no era la misma, daba un mensaje de error. Pero lo que pasaba era que se le cerraba la sesión al usuario. Y lo que pasaba era que la pagina se recargaba con la sesión sin iniciar. Y para solucionarlo era hacer que el formulario se renderizara
- Un error que llegue a tener es el siguiente, un error de guardar las imágenes de las rutas. En este caso fue un leve error que llegue a tener. Cuando el usuario añadía una o varias imágenes a una ruta, lo que sucedía era que en controlador de imagesRutaController.js,

en la función que añadía una nueva imagen, había ingresado mal el nombre de la carpeta donde se guardaba y las imágenes no se me guardaban. Y para solucionarlo lo que hice fue poner bien el nombre de la carpeta

Análisis

Interpretación de los resultados obtenidos

Después de los objetivos principales que me propuse, voy a opinar de que están bien cumplidos. Ya que, en la web, hay un sistema de seguridad para los usuarios, se pueden añadir nuevas rutas con un formulario y un mapa interactivo haciendo que sea intuitivo para el usuario. Además de eso los usuarios pueden hacer rutas y añadir comentarios en ella. También se puede organizar quedadas y participar en ella, y tener un mapa interactivo para poder ve las rutas que tienes por tu zona

Comparación de los resultados con los objetivos iniciales del proyecto

El objetivo principal que era hacer que los usuarios pudieran añadir rutas mediante un formulario, y de hacer la ruta con un mapa interactivo usando las coordenadas se ha podido hacer de forma correcta. Ya que cuando un usuario agrega una ruta debe de rellenar un formulario, y luego se usa el api de Leaflet para poder hacer la ruta de forma correcta. Pero lo único que faltaba era que el mapa, calculara de manera correcta el desnivel positivo y negativo, y la altura máxima y mínima.

También el objetivo de poder hacer quedadas y apuntarse a ellas se puede marcar como completado. Ya que en la web los usuarios pueden llegar a organizar quedadas de rutas, y otros usuarios pueden llegar a apuntarse a ella. Lo único que faltaría, sería hacer que cuando los usuarios se apunten en la quedada, deban de rellenar un formulario para decir cuantas personas van, y los que organizan la quedada sepan los miembros que van.

El objetivo de poder reportar rutas este hecho, ya que, si los usuarios ven que el detalle de la ruta esta mal, porque se puede reportar de forma correcta. Aunque lo ideal hubiera sido de implementar la opción de reporte, además del mensaje que hay que ingresar en la ruta.

El objetivo marcar la ruta como hecha y añadir un comentario ha sido hecho a la perfección, ya que los usuarios pueden marcar las rutas como completadas y además de eso se pueden añadir comentarios con imágenes.

También el objetivo de poder tener un listado de rutas funciona bien, y de poder filtrar esas rutas esta logrado de manera correcta.

Evaluación de la eficacia del software desarrollado

En esta web yo creo que es bastante intuitiva, ya que en la barra de navegación se puede ver de forma correcta las opciones que tiene el usuario, por ejemplo, de ver las rutas donde tiene el botón de añadir una nueva ruta y tiene filtros para buscar por preferencia la ruta. También la opción de quedadas, donde solo aparecen las próximas quedadas y no hay las quedadas pasadas. Y además el mapa gps que hay, es bastante fácil de entender, ya que cuando se carga se centra en la ubicación exacta del usuario, y también se pueden llegar a apreciar los puntos donde empieza una ruta. Y cada usuario tiene un perfil fácil de entender, con sus rutas hechas, las que han creado y las que tienen en favoritas, y las quedadas organizadas y en las que has participado.

Además, puedo llegar a opinar que el acceso a la base de datos funciona correctamente, ya que todas las sentencias que se hacen en la web funcionan perfectamente.

Identificación de áreas de mejora

Algunas áreas de mejora que podría tener son las siguientes:

- Hacer un sistema de notificaciones, para notificar al usuario de quien se ha apuntado a la quedada que ha organizado, o también saber que usuarios han hecho tu ruta.
- Mejorar el sistema de reportes, añadiendo una opción del tipo de reportes para que haya diferentes tipos de reportes. Para poder mejorar la gestión y saber cuáles reportes son más importante.

Conclusiones

A lo largo del desarrollo del proyecto, he conseguido una web funcional que está bien orientada al senderismo. Lo que permite a los usuarios el poder explorar y crear rutas, así como organizar quedadas para poder fomentar una interacción entre varias personas para poder juntar a personas con los mismos intereses.

Al realizar este proyecto, me ha supuesto un importante aprendizaje tanto a nivel técnico como personal. Ya que, desde un punto de vista técnico, me ha permitido descubrir y trabajar con herramientas que desconocía hasta el momento, como la extensión de PostGIS para PostgreSQL, ya que me abre muchas posibilidades para futuros proyectos que requieran trabajar con datos geoespaciales. Al mismo tiempo el poder integrar, el api de Leaflet para tener un mapa interactivo y poder hacer el mapa de las rutas ha resultado bastante útil. Y el api de Nominatim para la geocodificación automática para las direcciones de las rutas ha resultado bastante útil. Y al usar estas herramientas me han permitido poder consolidar los conocimientos en el fullstack utilizando NodeJS y PostgrSQL.

En cuanto al resultado final, se puede afirmar que el proyecto se ha cumplido e incluso superado las expectativas iniciales. Ya que en un principio la idea era permitir a los usuarios registrar el punto de inicio de una ruta, sin embargo, durante el desarrollo surgió la posibilidad de integrar un mapa interactivo para poder dibujar el trazado completo, lo que permite mejorar la experiencia del usuario. Del mismo modo, la funcionalidad de quedadas, propuesta por la tutora del proyecto, resultó ser una aportación muy valiosa ya que añade una dimensión social y comunitaria a la plataforma que la diferencia de otras aplicaciones similares del mercado.

Se puede que este proyecto, es el resultado de un trabajo que me ha permitido aplicar de forma práctica los conocimientos que se han adquirido a lo largo del primer y segundo año de DAM. Y de poder seguir adquiriendo nuevos conocimientos para futuros proyectos

Líneas de investigación futuras

De cara al futuro estaría bien hacer estas mejoras

- Hacer que la web tenga un sistema de seguidores para permitir a los usuarios seguirse entre ellos, y crear esta web en una red social
- Hacer que esta web sea disponible para dispositivos móviles, hacer este proyecto para móvil

- Añadir un sistema de logros, como por rutas hechas, o quedadas participado, o rutas creadas.
- Tener un sistema de estrellas al hacer rutas, diciendo al usuario que agregue una puntación de que opina la ruta
- Permitir que los usuarios hagan sus rutas mientras van avanzado, seria añadir una opción que vaya pillando el móvil por ubicación y que le permita hacer la ruta. Seria como hacer un strava.

Bibliografía

- Anwar, A. (2024, February 14). *OpenStreetMap OSM Nominatim API tutorial*. The Afi Labs Blog. https://blog.afi.io/blog/openstreetmap-osm-nominatim-api-tutorial/
- *Bootstrap icons*. (n.d.). Getbootstrap.com. Retrieved May 18, 2026, from https://icons.getbootstrap.com/
- *ChatGPT*. (n.d.). ChatGPT. Retrieved May 25, 2026, from https://chatgpt.com/
- *Claude*. (n.d.). Claude. Retrieved May 25, 2026, from https://claude.ai/chat/555075f8-6a95-4681-aaec-23c537cb83fa
- Codelixir [@codelixir1123]. (n.d.). *Geolocalizacion con postgresql & nodejs* [Video]. Youtube. Retrieved May 18, 2026, from https://www.youtube.com/watch?v=yK8wK2O7Ge0
- *¿Cómo usar bases de datos Postgres con Node.js?* (n.d.). ..Ed.team. Retrieved May 18, 2026, from https://ed.team/blog/como-usar-bases-de-datos-postgres-con-nodejs
- CorneGramm Desarrollo web-Web development [@cornegrammdesarrolloweb-we9919]. (n.d.). *Crear CRUD con Nodejs y Mysql - 2022* [Video]. Youtube. Retrieved May 18, 2026, from https://www.youtube.com/watch?v=lSdg8rQd4Ms
- *ES:Nominatim*. (n.d.). Openstreetmap.org. Retrieved May 18, 2026, from https://wiki.openstreetmap.org/wiki/ES:Nominatim
- *HTML tutorial*. (n.d.). W3schools.com. Retrieved May 18, 2026, from https://www.w3schools.com/html/default.asp
- *Leaflet — an open-source JavaScript library for interactive maps*. (n.d.). Leafletjs.com. Retrieved May 18, 2026, from https://leafletjs.com/
- *Login*. (n.d.). Figma. Retrieved May 18, 2026, from https://www.figma.com/login?is_not_gen_0=true&resource_type=team
- Luis, P. [@LuisSubiabre]. (n.d.). *Descargar e Instalar postgreSQL 17 en Windows 11* [Video]. Youtube. Retrieved May 18, 2026, from https://www.youtube.com/watch?v=dxNDNMD3RMQ
- Otto, M., & Thornton, J. (n.d.). *Bootstrap*. Getbootstrap.com. Retrieved May 18, 2026, from https://getbootstrap.com
- *Quick Start Guide - Leaflet - a JavaScript library for interactive maps*. (n.d.). Leafletjs.com. Retrieved May 18, 2026, from https://leafletjs.com/examples/quick-start/
- Robert, K. (2020, November 7). *Upload and Download Images to a Postgres DB (node.js)*. Medium. https://robert-keller22.medium.com/upload-and-download-images-to-a-postgres-db-node-js-92e43f232ae4
- Skelly, C. (2021, April 12). *Bootstrap 5 sidebar examples*. DEV Community. https://dev.to/codeply/bootstrap-5-sidebar-examples-38pb
- *Visorando : idées de randonnée pédestre, VTT, raquette*. (n.d.). Visorando. Retrieved May 18, 2026, from https://www.visorando.com/
- (N.d.-a). Youtube.com. Retrieved May 18, 2026, from https://www.youtube.com/watch?v=6OWxeUBOo2c
- (N.d.-b). Youtube.com. Retrieved May 18, 2026, from https://www.youtube.com/watch?v=YzVYI1TxVFc
- (N.d.-c). Youtube.com. Retrieved May 18, 2026, from https://www.youtube.com/watch?v=oVEUcYKemXQ

- (N.d.-d). Youtube.com. Retrieved May 18, 2026, from https://www.youtube.com/watch?v=Exf_oKPRr4g&list=PL_YyCdnLDJAjD4sfB3z2p_MOcleKUCVwy&index=2
- (N.d.-e). Dongee.com. Retrieved May 18, 2026, from https://www.dongee.com/tutoriales/tutorial-html-como-poner-colores-degradados-en-html-de-forma-profesional/
- (N.d.-f). Npmjs.com. Retrieved May 18, 2026, from https://www.npmjs.com/package/node-nominatim2
- (N.d.-g). Youtube.com. Retrieved May 18, 2026, from https://www.youtube.com/watch?v=YfKSmfG9Nro
- (N.d.-h). Wix.com. Retrieved May 18, 2026, from https://manage.wix.com/account/websites?referralAdditionalInfo=Route
- (N.d.-i). Wikiloc.com. Retrieved May 18, 2026, from https://es.wikiloc.com/
- (N.d.-j). Alltrails.com. Retrieved May 18, 2026, from https://www.alltrails.com/es
- (N.d.-k). Youtube.com. Retrieved May 18, 2026, from https://www.youtube.com/watch?v=Zy89Nj7tNNM
- (N.d.-l). Mappinggis.com. Retrieved May 18, 2026, from https://mappinggis.com/2020/01/como-calcular-distancias-con-leaflet-y-otros-metodos-de-conversion/

Pablo Martin Tomasi 2ºDAM

Fdo: Pablo Martín Tomasi

_____

Fdo: Amelia

_____